

Instituto Federal Catarinense
Curso de Bacharelado em Engenharia Elétrica
Campus Blumenau

Arthur Pedro Ferreira

Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0

Blumenau
2026

Arthur Pedro Ferreira

Arquitetura de um Sistema Supervisório Orientada a Serviços para a Indústria 4.0

Trabalho de Conclusão de Curso submetido ao Curso Superior de Bacharelado em Engenharia Elétrica do Instituto Federal Catarinense - Campus Blumenau, para a obtenção do título de Bacharel em Engenharia Elétrica.

Orientador: Prof. Dr. Vitor Mateus Moraes
Coorientador: Prof. Dr. Hylson Vescovi Netto

Blumenau
2026

No meio da dificuldade, encontra-se a oportunidade.
Albert Einstein

Resumo

Os sistemas supervisórios (*Supervisory Control and Data Acquisition*, SCADA) monitoram, supervisionam e controlam processos industriais em tempo real, reunindo em uma única interface a leitura das variáveis de campo, o registro histórico, a sinalização de alarmes e o envio de comandos aos equipamentos. Os sistemas convencionais, porém, exigem licenças proprietárias, hardware dedicado e instalação local especializada, o que os mantém fora do alcance de pequenas e médias empresas. Diante disso, este trabalho apresenta o desenvolvimento e a validação de um sistema supervisório com arquitetura orientada a serviços, acessado pelo navegador, sem instalação local. A arquitetura separa o *front-end*, o *back-end*, os *workers*, que processam as filas de trabalho, o serviço de aquisição Modbus e um executor isolado de código do usuário em JavaScript. O PostgreSQL guarda o cadastro e o histórico, o Redis mantém o valor atual de cada ponto e as filas de trabalho, e os serviços se comunicam pelo Moleculer. O acesso é controlado por cargo e por grupo de permissões, e cada comando é validado pela resposta do equipamento e fica registrado. São revisados os conceitos de sistemas supervisórios, Indústria 4.0, computação em nuvem e contêineres que fundamentam a proposta. A validação combina testes automatizados e uma bancada com controlador lógico programável TM221 e inversor de frequência ATV320, em que o sistema lê as grandezas do motor, sinaliza alarmes e envia comandos ao equipamento. A implantação em um provedor de nuvem e em mais de um servidor fica como trabalho futuro.

Palavras-chave: SCADA; supervisão de processos industriais; monitoramento remoto; contêineres; arquitetura orientada a serviços; Indústria 4.0; Modbus.

Abstract

Supervisory control and data acquisition (SCADA) systems monitor, supervise and control industrial processes in real time, bringing together in a single interface the reading of field variables, historical logging, alarm signaling and the dispatch of commands to the equipment. Conventional systems, however, require proprietary licenses, dedicated hardware and specialized local installation, which keeps them out of reach for small and medium-sized enterprises. In view of this, this work presents the development and validation of a supervisory system with a service-oriented architecture, accessed through the browser with no local installation. The architecture separates the front end, the back end, the workers, which process the work queues, the Modbus acquisition service and an isolated executor for user code written in JavaScript. PostgreSQL stores the registry and the history, Redis keeps the current value of each point and the work queues, and the services communicate through Moleculer. Access is controlled by role and by permission group, and each command is validated by the response of the equipment and recorded. The concepts of supervisory systems, Industry 4.0, cloud computing and containers that underpin the proposal are reviewed. The validation combines automated tests and a test bench with a TM221 programmable logic controller and an ATV320 variable frequency drive, in which the system reads the motor quantities, signals alarms and sends commands to the equipment. Deployment on a cloud provider and on more than one server is left as future work.

Keywords: SCADA; industrial process supervision; remote monitoring; containers; service-oriented architecture; Industry 4.0; Modbus.

Lista de Figuras

1 Linha do Tempo das Revoluções Industriais . . . 10
2 Pirâmide de Automação . . . 11
3 Tela de Supervisão . . . 14
4 Estrutura de um Sistema Ciber-Físico . . . 16
5 Interseções de IoT, CPS, IIoT e Indústria 4.0 . . . 17
6 Máquinas virtuais e contêineres . . . 18
7 Fluxo de uma fila de mensagens com tentativas e tarefas falhas . . . 20
8 Arquitetura do Sistema . . . 25
9 Mapa dos blocos usados nos fluxos . . . 31
10 Fluxo do cadastro de uma tag . . . 32
11 Fluxo da leitura do equipamento até o banco . . . 33
12 Fluxo da exibição do valor atual por consulta periódica . . . 34
13 Fluxo da consulta de tendência . . . 34
14 Fluxo do comando . . . 35
15 Fluxo da detecção, da sinalização e do reconhecimento de um alarme . . . 36
16 Fluxo do cálculo de uma variável meta . . . 37
17 Casco da aplicação com menu lateral e indicador de alarmes . . . 40
18 Mapa das plantas com o estado de alarme de cada uma . . . 41
19 Dashboard em execução com painéis de linha, barra e medidor . . . 42
20 Editor do sinótico com paleta, área de desenho e inspetor . . . 43
21 Sinótico em execução com valores ao vivo, selos de alarme e a lista de valores da tela . . . 44
22 Lista de acompanhamento com valores ao vivo e gráfico . . . 45
23 Diálogo de comando na confirmação (acima) e no resultado (abaixo) . . . 46
24 Formulário de configuração de um alarme . . . 47
25 Tela de alarmes ativos com reconhecimento . . . 48
26 Linha do tempo com eventos de alarme e de dispositivo . . . 49
27 Tendência de uma tag em janela de sete dias . . . 50
28 Formulário de uma tag analógica Modbus . . . 51
29 Grade de tags com valor ao vivo . . . 52
30 Tela de usuários . . . 53
31 Formulário de grupo com o catálogo de permissões . . . 54
32 Editor de código da variável meta com o resultado do botão Testar . . . 55
33 Bancada de teste montada. . . . 57
34 Topologia de comunicação da bancada. . . . 58
35 Ligação serial RS-485 entre o CLP e o inversor. . . . 58
36 Configuração da comunicação serial no EcoStruxure Machine Expert Basic. . . . 59
37 Parametrização do inversor ATV320. . . . 60
38 Blocos funcionais do programa do CLP. . . . 60
39 Telas de supervisão da bancada. . . . 62

Lista de Siglas

Sigla	**Significado**
API	*Application Programming Interface*
CLP	Controlador Lógico Programável
CNC	Controlador Numérico Computadorizado
CPS	Sistemas Ciber-físicos (*Cyber-Physical Systems*)
HTTP	*Hypertext Transfer Protocol*
IHM	Interface Homem-Máquina
IIoT	Internet Industrial das Coisas (*Industrial Internet of Things*)
I/O	Entrada/Saída (*Input/Output*)
IoT	Internet das Coisas (*Internet of Things*)
IP	*Internet Protocol*
ISA	*International Society of Automation*
JSON	*JavaScript Object Notation*
RAM	*Random Access Memory*
REST	*Representational State Transfer*
RS-485	
SCADA	Supervisão, Controle e Aquisição de Dados (*Supervisory Control and Data Acquisition*)
SOA	Arquitetura Orientada a Serviços (*Service-Oriented Architecture*)
SQL	*Structured Query Language*
TTL	*time-to-live*
URL	*Uniform Resource Locator*

Lista de Tabelas

1 Requisitos Funcionais. 23
2 Requisitos Não Funcionais. 24
3 Serviços e responsabilidades. 28
4 Efeito da falha de cada bloco nos fluxos do sistema. 38
5 Resumo dos fluxos do sistema. 39
6 Telas, cargos que acessam e requisitos atendidos. 55
7 Equipamentos da bancada de teste. 57
8 Parâmetros do ATV320. 59
9 Mapa de memória do CLP. 60
10 Cadastro da bancada no sistema. 61
11 Tags cadastradas no sistema para a bancada. 61
12 Ensaios da bancada. 63
13 Medidas previstas nos ensaios da bancada. 64

Índice

1 Introdução **10**
1.1 Contextualização . . . 10
1.2 Justificativa . . . 11
1.3 Objetivos . . . 12
1.3.1 Objetivo Geral . . . 12
1.3.2 Objetivos Específicos . . . 12
1.4 Metodologia . . . 13

2 Conceitos e Tecnologias Utilizadas **14**
2.1 Sistema SCADA . . . 14
2.1.1 Aquisição e Registro de Dados . . . 14
2.1.2 Sincronismo de Tempo . . . 15
2.1.3 Níveis de Acesso . . . 15
2.1.4 Comando e Controle . . . 15
2.1.5 Escalabilidade . . . 15
2.1.6 Integrabilidade . . . 15
2.1.7 Redundância . . . 16
2.2 Automação e Tecnologias da Indústria 4.0 . . . 16
2.3 Computação em Nuvem . . . 17
2.4 Contêineres . . . 18
2.5 Arquitetura Orientada a Serviços . . . 18
2.6 HTTP, API REST e JSON . . . 18
2.7 Node.js, TypeScript e Fastify . . . 19
2.8 Moleculer . . . 19
2.9 Filas de Mensagens e Processamento Assíncrono . . . 20
2.10 Banco de Dados . . . 20
2.11 Redis . . . 21
2.12 React e Next.js . . . 21
2.13 Execução Isolada de Código . . . 21
2.14 Protocolo Modbus . . . 22

3 Arquitetura e Requisitos do Sistema **23**
3.1 Requisitos . . . 23
3.1.1 Requisitos Funcionais . . . 23
3.1.2 Requisitos Não Funcionais . . . 24
3.2 Arquitetura . . . 25
3.3 Validações dos Requisitos . . . 26
3.3.1 Aquisição e Registro de Dados . . . 26
3.3.2 Sincronismo de Tempo . . . 26
3.3.3 Níveis de Acesso . . . 26
3.3.4 Comando e Controle . . . 26
3.3.5 Escalabilidade . . . 27
3.3.6 Integrabilidade . . . 27
3.3.7 Redundância . . . 27
3.4 Serviços . . . 28
3.4.1 Front-end . . . 28

3.4.2 Back-end . . . 28
3.4.3 Worker . . . 28
3.4.4 Aquisição . . . 29
3.4.5 Executor de Scripts . . . 29
3.4.6 PostgreSQL . . . 29
3.4.7 Redis . . . 30
3.4.8 Comunicação Moleculer e Filas . . . 30

4 Fluxos do Sistema **31**
4.1 Cadastro de uma Tag . . . 31
4.2 Do Campo ao Banco . . . 32
4.3 Do Banco à Tela em Tempo Real . . . 33
4.4 Histórico e Tendência . . . 34
4.5 Comando de Equipamentos . . . 35
4.6 Alarme . . . 36
4.7 Variável Meta . . . 37
4.8 Efeito das Falhas . . . 38
4.9 Resumo dos Fluxos . . . 38

5 Interface do Sistema **40**
5.1 Estrutura geral da aplicação . . . 40
5.2 Mapa das plantas . . . 40
5.3 Dashboard . . . 41
5.4 Sinótico . . . 42
5.5 Lista de acompanhamento . . . 44
5.6 Diálogo de comando . . . 45
5.7 Alarmes e eventos . . . 46
5.8 Linha do tempo . . . 48
5.9 Histórico e tendência . . . 49
5.10 Cadastro: plantas, dispositivos de E/S, tags e pastas . . . 50
5.11 Usuários, cargos e grupos . . . 52
5.12 Variável meta e scripts . . . 54
5.13 Síntese: telas, cargos e requisitos . . . 55

6 Testes e Resultados **57**
6.1 Objetivo e Visão Geral . . . 57
6.2 Equipamentos Utilizados . . . 57
6.3 Arquitetura de Comunicação . . . 58
6.4 Configuração do Inversor ATV320 . . . 59
6.5 Programa do Controlador . . . 60
6.6 Configuração no Supervisório . . . 61
6.7 Ensaios e Resultados . . . 63

7 Conclusão **66**
7.1 Retomada dos objetivos . . . 66
7.2 Contribuições . . . 66
7.3 Limitações . . . 66
7.4 Trabalhos futuros . . . 67

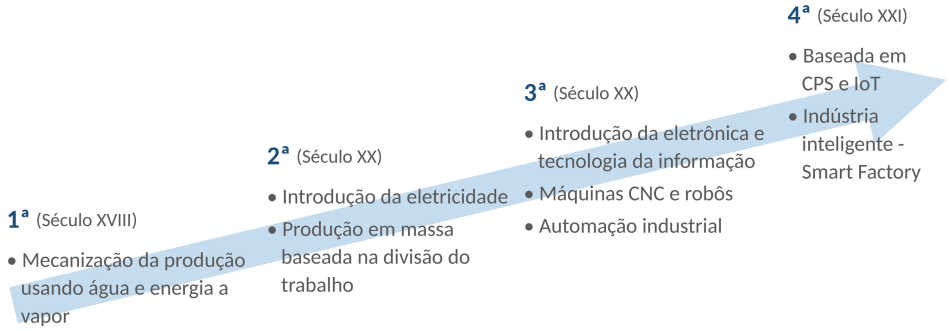
1 Introdução

1.1 Contextualização

A indústria é o setor da economia dedicado à produção de bens materiais, apoiado em trabalho humano, energia e equipamentos de alto valor agregado. Ao longo de sua evolução, sucessivos saltos tecnológicos redefiniram processos, produtos e formas de organização do trabalho, as chamadas "revoluções industriais"(Lasi *et al.*, 2014).

A Figura 1 resume essa linha do tempo. Na primeira fase, destaca-se a mecanização da produção, na segunda, a difusão da energia elétrica e a produção em massa e, na terceira, a digitalização (Lasi *et al.*, 2014; Pisching, 2018). Foi nessa fase que a introdução da tecnologia da informação, da automação e da eletrônica impulsionou o início da automação dos processos produtivos. Surgiram então os controladores lógicos programáveis (CLPs), os Controladores Numéricos Computadorizados (CNCs) e os robôs (Pisching, 2018). Essas tecnologias tornaram-se essenciais para as indústrias que buscam maior eficiência e qualidade com menores custos e prazos. Sua adoção contribui diretamente para o aumento da produtividade (Bigheti, 2020).

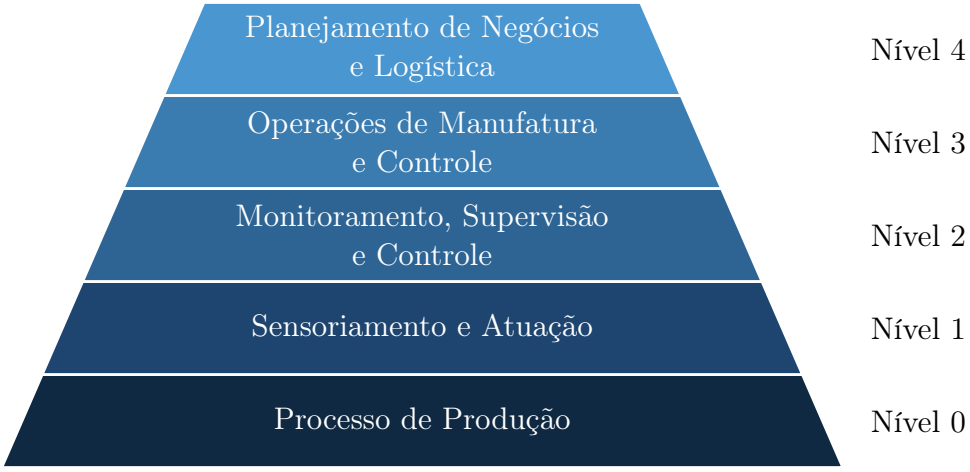
Figura 1: Linha do Tempo das Revoluções Industriais



Fonte: Adaptado de Pisching (2018).

Atualmente, vive-se a quarta fase, a chamada Indústria 4.0, baseada na indústria inteligente, na Internet das Coisas (*Internet of Things*, IoT) e nos sistemas ciber-físicos (*Cyber-Physical Systems*, CPS) (Bigheti, 2020). Nesses sistemas, subsistemas computacionais cooperam entre si e permanecem integrados ao mundo físico e a seus processos, o que permite medir, controlar e analisar dados em tempo real por meio da rede, inclusive pela Internet (Monostori *et al.*, 2016).

Figura 2: Pirâmide de Automação



Fonte: Adaptado de Bangemann *et al.* (2014).

A Figura 2 organiza a automação na Indústria 4.0 em forma de pirâmide, distribuída em cinco níveis. No nível 0 está o processo físico de produção, com elementos como motores, bombas e tanques. No nível 1, o sensoriamento e a atuação, realizados por sensores, transmissores e válvulas. O nível 2 reúne o monitoramento, a supervisão e o controle, camada em que se encontram os controladores lógicos programáveis (CLPs), os sistemas supervisórios (SCADA) e as interfaces homem-máquina (IHM), responsáveis por acompanhar e comandar o processo em tempo real e por se comunicar com o nível 1 (Bangemann *et al.*, 2014). O nível 3 corresponde às operações de manufatura e ao controle da produção. No topo, o nível 4 abrange o planejamento de negócios e a logística (Bigheti, 2020).

Recentemente, as camadas do nível 2 ao nível 4 ganharam notoriedade: serviços antes restritos à planta passaram a ser oferecidos em nuvem e compartilhados entre sistemas e usuários, em geral segundo a arquitetura orientada a serviços (*Service-Oriented Architecture*, SOA) (Bangemann *et al.*, 2014; Abbas, 2011). Esse movimento é sustentado pela Indústria 4.0, com o avanço dos protocolos de comunicação e de telemetria, a popularização do software e a interconexão de máquinas e sistemas (Lasi *et al.*, 2014; Pisching, 2018; Bigheti, 2020).

Esses serviços se apoiam hoje em contêineres. Um contêiner empacota uma aplicação com suas dependências e a executa de forma isolada, o que permite dividir um sistema em serviços independentes (Netto, Hylson Vescovi *et al.*, 2017; Netto, Hylson V. *et al.*, 2016), e executa da mesma forma em um computador local, em um servidor ou em um provedor de nuvem.

Nesse cenário, este trabalho propõe, implementa e valida um sistema supervisório com arquitetura orientada a serviços, que disponibiliza pelo navegador funcionalidades antes restritas ao ambiente físico da planta (Bangemann *et al.*, 2014). O sistema é executado em contêineres num servidor, e a mesma arquitetura poderia ser levada a um provedor de nuvem. Alinhada à Indústria 4.0 (Bigheti, 2020), a proposta torna o monitoramento e a supervisão acessíveis e compartilháveis entre sistemas e usuários.

1.2 Justificativa

Os processos industriais geram grande volume de dados, apresentados ao operador por um software de supervisão, por meio de uma interface homem-máquina (IHM), que converte leituras de campo em telas, tendências e alarmes (Zanghi, 2019). Com computadores mais

capazes, protocolos melhores e a integração com a Internet, consolidou-se a supervisão remota. Em setores como saneamento, transporte e energia, com instalações espalhadas, ganham peso a telemetria e o controle à distância. Uma arquitetura típica de SCADA integra os equipamentos de campo, como CLPs, aos módulos de monitoramento e controle e centraliza a visualização para o operador (Abbas, 2011).

Os supervisórios convencionais costumam exigir licenças proprietárias, hardware dedicado e infraestrutura local, o que os mantém fora do alcance de pequenas e médias empresas (Phuyal *et al.*, 2020). Oferecer a supervisão pelo navegador reduz esse investimento e amplia o monitoramento a instalações dispersas, com menos deslocamentos e resposta mais ágil a ocorrências (Abbas, 2011).

A computação em nuvem oferece armazenamento, processamento, bancos de dados e aplicações como serviços remotos (Pedrosa; Nogueira, 2011), com capacidade sob demanda, o que reduz o investimento local (Henriques da Silva, 2010). A conectividade, em especial sem fio, permite que dispositivos e sistemas ciber-físicos troquem dados com a nuvem (Stankovic, 2014; Monostori *et al.*, 2016).

Neste trabalho, a nuvem é uma possibilidade de aplicação, e não a forma de implantação adotada. O sistema roda em contêineres num servidor, o que dispensa contratos de hospedagem e mantém baixo o custo, requisito central para pequenas e médias empresas. Como cada serviço é um contêiner independente, a solução poderia ir a um provedor de nuvem sem alteração de código (Netto, Hylson Vescovi *et al.*, 2017; Netto, Hylson V. *et al.*, 2016), e o acesso pelo navegador dispensa instalar programas nos computadores dos usuários.

1.3 Objetivos

A seguir são apresentados os objetivos gerais e específicos deste Trabalho de Conclusão de Curso.

1.3.1 Objetivo Geral

Desenvolver e validar um sistema supervisório com arquitetura orientada a serviços em contêineres, capaz de apoiar o monitoramento, o comando e a supervisão de processos industriais, com serviços bem definidos, reutilizáveis e acessíveis pelo navegador.

1.3.2 Objetivos Específicos

- Revisar os conceitos de Indústria 4.0, sistemas supervisórios, arquitetura orientada a serviços (SOA), contêineres e computação em nuvem, de modo a embasar a solução proposta.
- Especificar os requisitos funcionais e não funcionais e projetar a arquitetura de serviços em contêineres, com a separação entre *front-end*, *back-end*, serviços de aquisição, filas de trabalho e armazenamento de dados.
- Implementar a aquisição de dados via Modbus, com agendamento das leituras e escritas, e o registro do valor atual e do histórico das variáveis.
- Implementar a interface de supervisão, com telas de tempo real, alarmes, gráficos de tendência e sinótico.
- Implementar o comando de equipamentos e a validação do seu resultado.
- Implementar o controle de acesso por cargo e por grupo de permissões.

- Validar o sistema em uma bancada de teste com um controlador lógico programável (CLP) e um inversor de frequência.

1.4 Metodologia

O trabalho foi feito em quatro etapas: revisão bibliográfica, especificação do sistema, implementação e validação. Os próximos parágrafos indicam em quais capítulos cada etapa é apresentada.

A revisão bibliográfica abrange sistemas supervisórios (SCADA), Indústria 4.0, IoT e CPS, além de SOA, contêineres, computação em nuvem, bancos de dados, filas de trabalho e o protocolo Modbus, apresentados no Capítulo 2.

O Capítulo 3 especifica os requisitos funcionais e não funcionais e a arquitetura, com a divisão em serviços em contêineres e a separação entre *front-end*, *back-end*, aquisição e armazenamento.

O Capítulo 4 descreve os fluxos do sistema: a aquisição Modbus, o agendamento das leituras, o registro do valor atual e do histórico, os alarmes, a variável meta e o comando de equipamentos.

O Capítulo 5 apresenta a interface: o mapa das plantas, o dashboard, o sinótico, as telas de acompanhamento em tempo real, de alarmes, de linha do tempo e de histórico, e as telas de cadastro e de usuários e grupos.

Na validação, o Capítulo 6 descreve a bancada de teste, com um CLP e um inversor de frequência, e os ensaios com seus resultados. O Capítulo 7 reúne as conclusões e as perspectivas de evolução.

2 Conceitos e Tecnologias Utilizadas

Este capítulo reúne os conceitos e as tecnologias que sustentam o trabalho, do supervisório e da automação à infraestrutura de execução, aos dados, às tecnologias de aplicação e de segurança e ao protocolo de comunicação com o campo.

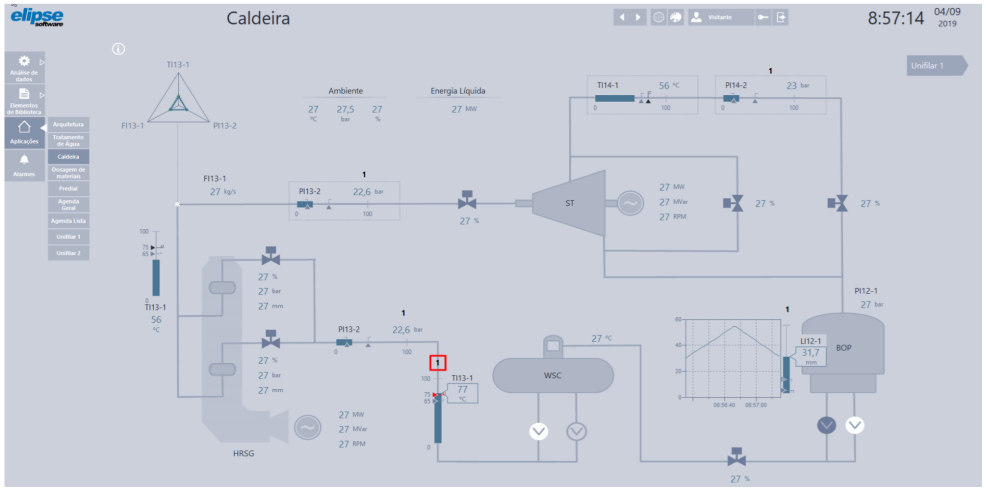
2.1 Sistema SCADA

Com a queda dos custos de computadores e de equipamentos de rede e o avanço do software (Abbas, 2011), as empresas de automação industrial passaram a oferecer soluções genéricas, que atendem a diferentes processos e protocolos de comunicação. Essas ferramentas receberam o nome de SCADA, sigla em inglês para *Supervisory Control and Data Acquisition* (Zanghi, 2019).

Na prática, o SCADA é um programa que coleta e guarda os dados de um processo industrial e permite acompanhar e controlar esse processo, no local ou à distância (Zanghi, 2019; Phuyal *et al.*, 2020). Ele não é feito para um único processo. Com objetos prontos, o mesmo programa pode ser configurado para uma indústria alimentícia, petroquímica ou têxtil, por exemplo, e se comunicar com diferentes equipamentos e protocolos industriais (Zanghi, 2019).

O SCADA é usado principalmente em processos contínuos e espalhados por vários locais, e muitos deles são serviços essenciais, as chamadas infraestruturas críticas (Yadav; Paul, 2020). Os usos típicos incluem energia elétrica, água e saneamento, óleo e gás, transportes, indústrias química e farmacêutica, papel e celulose, alimentos e bebidas e manufatura (Stouffer *et al.*, 2015). A Figura 3 ilustra uma tela de supervisão.

Figura 3: Tela de Supervisão



Fonte: Elipse Software (2021).

Entre os conceitos de um sistema SCADA, sete orientam este trabalho: aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade, integrabilidade e redundância (Zanghi, 2019).

2.1.1 Aquisição e Registro de Dados

Atualmente, predominam dispositivos inteligentes, que medem grandezas analógicas, como uma temperatura, e digitais, como uma bomba ligada ou desligada, e enviam esses valores ao

supervisório por uma rede de dados (Yadav; Paul, 2020). Cada ponto lido é chamado de *tag* e tem um tipo de dado, como booleano, inteiro, real ou texto. O supervisório guarda esses valores ao longo do tempo, o que permite analisar depois o que aconteceu, por gráficos de tendência e listas de eventos (Zanghi, 2019).

2.1.2 Sincronismo de Tempo

Todo valor coletado precisa vir acompanhado da data e da hora a que se refere. Isso se chama estampa de tempo. Sem isso, não seria possível saber em que ordem as coisas aconteceram ao investigar uma ocorrência ou uma falha (Zanghi, 2019).

Na prática, o supervisório pergunta o valor aos dispositivos de tempos em tempos (Stouffer *et al.*, 2015). O intervalo entre uma pergunta e outra se chama período de atualização e depende da rapidez do processo (Yadav; Paul, 2020). Por isso, a estampa marca o momento da leitura, e não o momento exato em que o valor mudou, e o registro não consegue ser mais preciso do que esse período.

2.1.3 Níveis de Acesso

Nem todo usuário pode fazer tudo no sistema. Os níveis de acesso definem o que cada pessoa pode acompanhar e comandar. Cada pessoa recebe um cargo, com apenas as permissões de que precisa para o seu trabalho, e precisa se identificar para entrar no sistema (Stouffer *et al.*, 2015).

2.1.4 Comando e Controle

Comandar é enviar uma ordem a um equipamento de campo, o que na prática é escrever um valor no dispositivo (Yadav; Paul, 2020). O comando pode ser instantâneo, enviado na hora, ou agendado, conforme situações definidas antes, com o horário registrado (Zanghi, 2019).

Para evitar erros, o operador confirma a ação antes de a ordem ir para o equipamento (Zanghi, 2019). Depois, o supervisório verifica se o dispositivo aceitou a ordem e, se houve falha, gera um alarme, pois um comando errado pode danificar equipamentos ou colocar em risco a segurança do processo (Stouffer *et al.*, 2015).

2.1.5 Escalabilidade

As instalações costumam crescer com o tempo. Uma aplicação, pode ganhar novos circuitos e equipamentos, e o SCADA precisa acompanhar esse crescimento. Essa capacidade se chama escalabilidade: deve ser possível acrescentar pontos à base de dados e desenhar novas telas de operação (Zanghi, 2019). Na prática, acrescentar um ponto deve ser uma configuração, e não uma mudança no programa. O sistema passa a ler o novo ponto sem que seja preciso alterar o código, instalar uma nova versão ou atrapalhar o que já está funcionando.

2.1.6 Integrabilidade

Todos os elementos de um sistema SCADA devem estar integrados, para que a supervisão e o controle funcionem sem problemas. Essa integração é feita pela comunicação de dados, com protocolos compatíveis. Os dispositivos de campo precisam se integrar ao supervisório, e ele também pode precisar trocar dados com outros programas (Zanghi, 2019).

2.1.7 Redundância

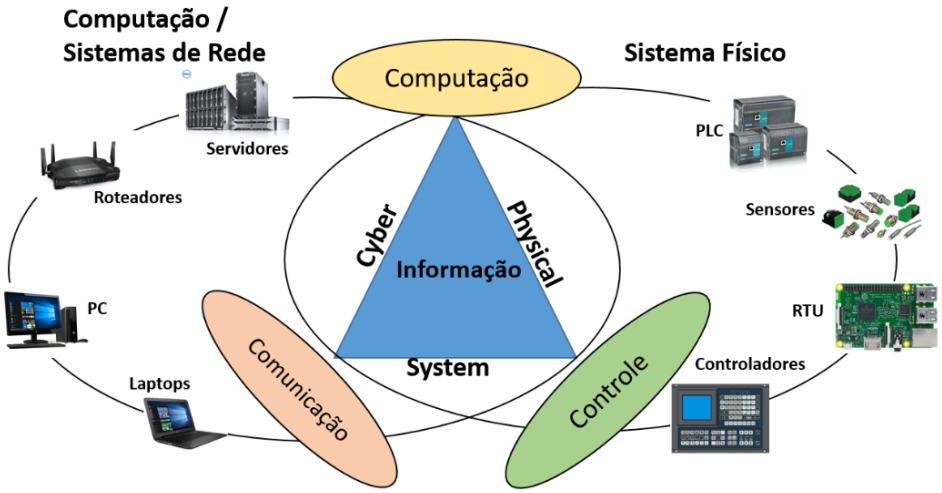
Redundância é ter um elemento reserva, idêntico ao original, que assume o trabalho quando o original falha, de preferência sem que o usuário perceba (Zanghi, 2019). Ela pode ser usada nos supervisórios, nos dispositivos de aquisição e controle e nos equipamentos de rede.

Em uma instalação típica, várias estações de operação se comunicam com os dispositivos de campo, e as unidades de aquisição e controle trabalham em pares, uma espelhando a outra (Zanghi, 2019). A recomendação é que cada componente crítico tenha um reserva, que as redes também sejam duplicadas e que o sistema continue supervisionando e controlando o processo mesmo quando uma parte falha (Stouffer *et al.*, 2015).

2.2 Automação e Tecnologias da Indústria 4.0

Os sistemas ciber-físicos (CPS) são sistemas com capacidade de computação ligados a dispositivos do mundo físico e aos processos ao seu redor, que ao mesmo tempo fornecem e usam serviços de dados disponíveis na internet (Bigheti, 2020; Monostori *et al.*, 2016). O termo foi proposto em 2006 nos Estados Unidos (Pisching, 2018). Nos sistemas de produção, três características se destacam: a inteligência, que é obter informações e agir de forma autônoma, a conectividade com outros elementos, pessoas e serviços, e a responsividade, que é reagir a mudanças (Monostori *et al.*, 2016). A Figura 4 ilustra essa estrutura.

Figura 4: Estrutura de um Sistema Ciber-Físico



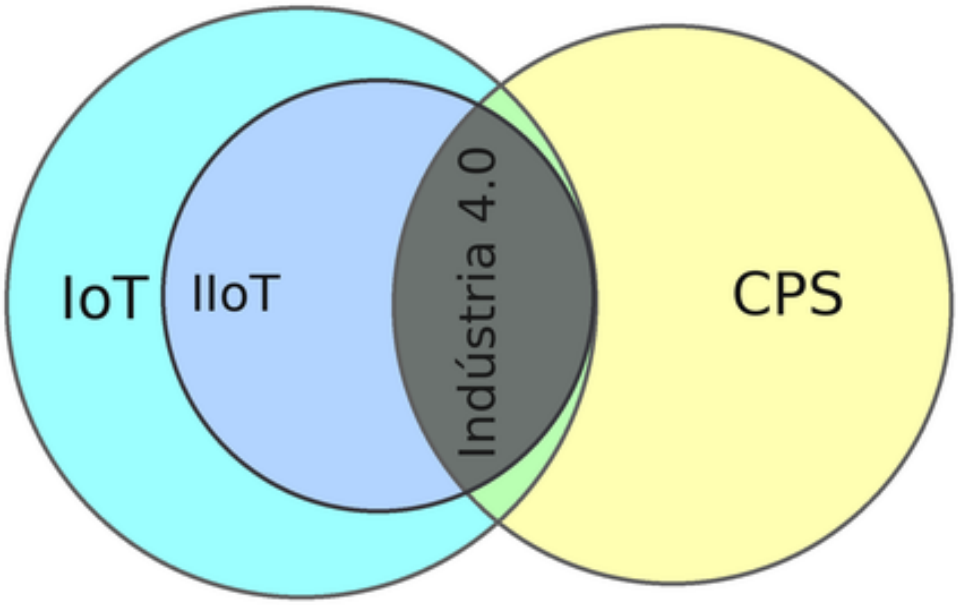
Fonte: Adaptado de Bigheti (2020).

À direita da figura está o sistema físico, com sensores, controladores e CLPs. À esquerda estão a computação e as redes, com servidores, roteadores e computadores. A comunicação leva os dados de um lado para o outro, e o controle faz o caminho de volta, agindo sobre o processo. Por exemplo, um sensor mede o nível de um tanque, e esse valor chega pela rede a um servidor. O servidor analisa o dado e pode enviar uma ordem de volta para ligar uma bomba.

Para que essa troca de dados aconteça, os objetos físicos precisam estar ligados em rede. É essa a ideia da Internet das Coisas (IoT), que liga objetos físicos, com sensores e atuadores, por meio de redes, principalmente sem fio, que se comunicam pela internet (Bigheti, 2020). Cada objeto tem uma identidade digital própria e pode trocar informações com pouca intervenção humana (Pisching, 2018; Bigheti, 2020). No ambiente industrial, ela recebe o nome de Internet Industrial das Coisas (*Industrial Internet of Things*, IIoT), voltada a aumentar a eficiência das

fábricas, com requisitos rígidos de tempo real, confiabilidade e segurança (Bigheti, 2020). A Figura 5 mostra como esses conceitos se relacionam: a IoT é o universo mais amplo, a IIoT é a parte dela aplicada à indústria, e a Indústria 4.0 surge do encontro entre a IIoT e os CPS.

Figura 5: Interseções de IoT, CPS, IIoT e Indústria 4.0



Fonte: Adaptado de Sisinni et al. (2018) apud Bigheti (2020, p. 27).

A Indústria 4.0 nasce da demanda por produtos personalizados e de produção descentralizada e do avanço da automação e da digitalização (Lasi *et al.*, 2014). Ela reúne CPS, IIoT, computação em nuvem, redes sem fio e uso intenso de dados, e sua ideia central é a fábrica inteligente (Bigheti, 2020). Seus princípios são a interoperabilidade, a digitalização, a descentralização, a coleta e análise de dados em tempo real, a orientação a serviços e a modularidade (Pisching, 2018).

À medida que os dispositivos de campo oferecem suas funções como serviços de rede, a pirâmide de automação da Figura 2, baseada na norma ISA-95 (International Society of Automation, 2000), tende a dar lugar a uma arquitetura plana baseada em serviços (Bangemann *et al.*, 2014). Nesse cenário, o SCADA, antes um sistema isolado, passa a fazer parte de uma rede de serviços (Yadav; Paul, 2020; Bigheti, 2020).

2.3 Computação em Nuvem

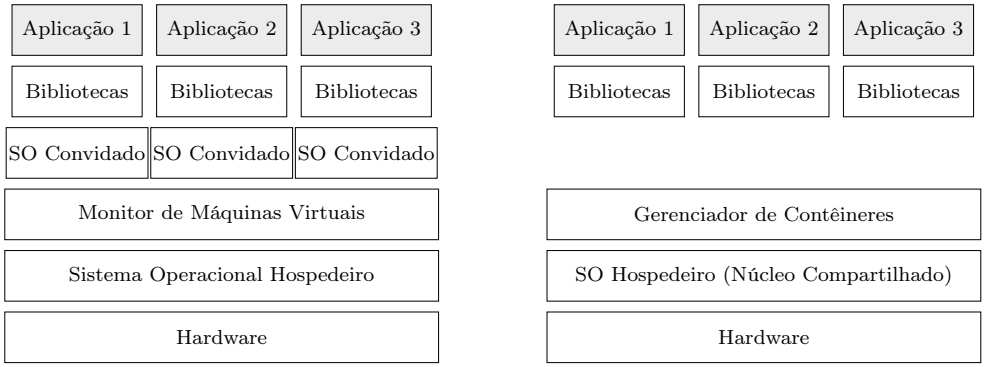
A computação em nuvem é um modelo em que dados e aplicações ficam em grandes centros de processamento, oferecidos como serviços pela internet (Henriques da Silva, 2010). Em vez de manter a própria infraestrutura, o usuário contrata de um provedor a capacidade de que necessita (Pedrosa; Nogueira, 2011). Entre as vantagens estão a dispensa de infraestrutura própria e a facilidade de ajustar a capacidade, e entre os desafios estão a segurança dos dados e a garantia de disponibilidade (Henriques da Silva, 2010; Pedrosa; Nogueira, 2011).

2.4 Contêineres

A virtualização oferece um ambiente virtual sobre os recursos físicos de um computador, para melhorar seu aproveitamento. Na forma tradicional, usa máquinas virtuais, cada uma com um sistema operacional completo, chamado de convidado, sob o controle de um hipervisor (Netto, Hylson Vescovi *et al.*, 2017). Esse modelo oferece forte isolamento entre as máquinas virtuais, mas cada uma carrega o custo de um sistema operacional inteiro (Netto, Hylson V. *et al.*, 2016).

O contêiner é uma alternativa mais leve: compartilha o núcleo do sistema hospedeiro e isola os seus processos dos demais, com um gerenciador que controla o uso dos recursos (Netto, Hylson Vescovi *et al.*, 2017). A ideia se difundiu com o Linux, cujo núcleo oferece o isolamento e a limitação de processador e memória (Docker, Inc., 2026). A Figura 6 compara os dois modelos.

Figura 6: Máquinas virtuais e contêineres



(a) Máquinas Virtuais (b) Contêineres

Fonte: Elaborado pelo autor com base em Hylson Vescovi Netto *et al.* (2017).

Sem inicializar um sistema operacional, o contêiner é criado muito mais rápido que uma máquina virtual (Netto, Hylson Vescovi *et al.*, 2017). Ele nasce de uma imagem, pacote imutável com a aplicação, suas bibliotecas e dependências, normalmente em camadas (Netto, Hylson V. *et al.*, 2016). A mesma imagem executa de forma idêntica no computador do desenvolvedor ou em um servidor. Em contrapartida, tudo o que o contêiner tem em memória ou em disco se perde quando ele é encerrado, a menos que seja gravado em volumes externos.

2.5 Arquitetura Orientada a Serviços

A arquitetura orientada a serviços (SOA) organiza um sistema como um conjunto de serviços que interagem entre si (Pisching, 2018). Cada serviço é um programa separado, que cuida de uma parte do trabalho e oferece suas funções aos demais pela rede. Essa organização facilita criar e reaproveitar serviços e traz modularidade, disponibilidade e escalabilidade às aplicações industriais. Como cada serviço é um componente separado, é possível aumentar a capacidade só da parte que precisa, sem ampliar o sistema inteiro (Bigheti, 2020).

2.6 HTTP, API REST e JSON

O protocolo HTTP (*Hypertext Transfer Protocol*) é a forma como os programas trocam informações na web: o cliente faz um pedido a um endereço (URL), e o servidor devolve os

dados e um código que indica o resultado. O padrão REST (*Representational State Transfer*) organiza APIs web de forma simples, com um método do HTTP para cada operação, como GET para consultar, POST para criar, PUT para atualizar e DELETE para remover. Ele é uma das maneiras de implementar serviços web, recomendados para concretizar a SOA (Pisching, 2018).

Os dados costumam vir no formato JSON (*JavaScript Object Notation*), um texto leve, fácil de ler por pessoas e máquinas. Em sistemas industriais, uma camada de software pode falar Modbus com o equipamento e expor seus dados por serviços REST (Bigheti, 2020).

O HTTP não guarda informação de um pedido para o outro, de modo que cada pedido leva o necessário para ser atendido, como a credencial do usuário. Os códigos de resposta 2xx indicam sucesso, os 4xx erro do cliente, como falta de autenticação (401) ou de permissão (403), e os 5xx erro do servidor.

Para acompanhar um dado que muda com o tempo, o servidor pode enviar as atualizações por uma conexão sempre aberta, como as *WebSockets*, ou o navegador pode consultá-lo em intervalos curtos. A consulta periódica é mais simples e funciona em qualquer infraestrutura HTTP, ao custo de mais pedidos e de um atraso que não pode ser menor que o intervalo de consulta.

2.7 Node.js, TypeScript e Fastify

O Node.js permite rodar JavaScript fora do navegador. Seu modelo é orientado a eventos, com operações de entrada e saída que não travam o programa (não bloqueantes): um único fluxo de execução atende muitas conexões, pois, enquanto uma operação de rede ou de disco aguarda, atende outros pedidos (OpenJS Foundation, 2026b). Em contrapartida, cálculos longos travam esse fluxo e devem ir para processos separados.

O TypeScript é uma extensão do JavaScript em que o programador declara o tipo de cada dado. O compilador verifica a coerência dos tipos e gera JavaScript comum, de modo que erros são detectados no desenvolvimento (Microsoft, 2026). Como os tipos só existem na compilação, dados que chegam de fora do programa precisam ser validados na execução.

O Fastify é um *framework* web para Node.js de baixo custo de processamento por pedido, organizado em módulos (*plugins*). Ele permite inserir rotinas próprias (*hooks*) nas etapas de cada pedido, como a autenticação e a validação, e descrever o formato do pedido e da resposta por esquemas (OpenJS Foundation, 2026a).

A validação por esquema descreve a estrutura que os dados devem ter e rejeita o que não a respeita, e uma única descrição pode servir na execução e na compilação (OpenJS Foundation, 2026a). Compartilhado entre servidor e navegador, o esquema mantém os dois de acordo sobre o formato das mensagens.

2.8 Moleculer

O Moleculer é uma ferramenta para construir sistemas de serviços em Node.js, que reúne em uma biblioteca leve o necessário para que os serviços se localizem na rede, dividam a carga e continuem operando diante de falhas (MoleculerJS, 2026).

Cada serviço expõe operações que outros podem chamar remotamente e envia avisos quando algo relevante acontece. Os serviços são descobertos automaticamente, sem hierarquia entre eles: cada um mantém o próprio registro dos demais e distribui as chamadas entre as cópias disponíveis (MoleculerJS, 2026; Bigheti, 2020). Para lidar com falhas, o Moleculer oferece

nova tentativa, tempo limite e resposta alternativa (MoleculerJS, 2026), capacidade alinhada às arquiteturas orientadas a serviços da Indústria 4.0 (Bigheti, 2020).

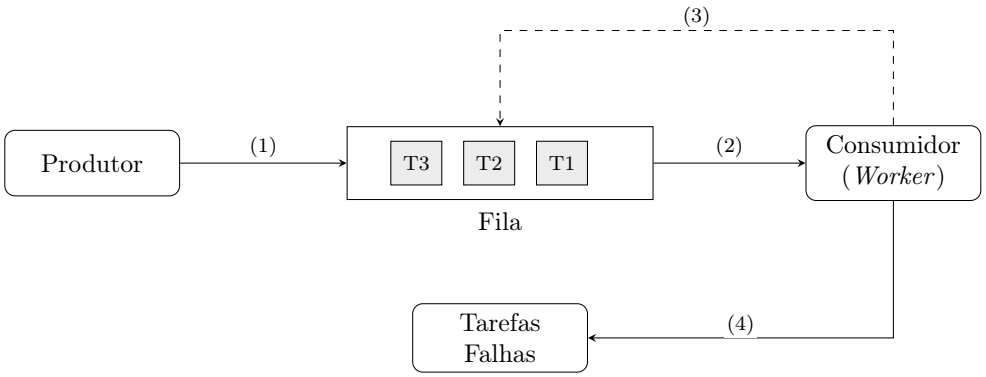
2.9 Filas de Mensagens e Processamento Assíncrono

Uma fila de mensagens separa quem pede um trabalho, o produtor, de quem o executa, o consumidor, frequentemente chamado de *worker*. A tarefa, ou *job*, fica na fila até haver capacidade, e o produtor segue adiante sem esperar o resultado, em processamento assíncrono (Taskforce.sh Inc., 2026).

A fila absorve picos de demanda, permite aumentar a capacidade acrescentando consumidores, isola falhas, pois o consumidor parado não impede o registro de novas tarefas, e tira do caminho de um pedido feito na tela os trabalhos demorados (Taskforce.sh Inc., 2026).

Como a fila pode entregar a mesma tarefa mais de uma vez, seu tratamento deve ser idempotente, isto é, executá-lo duas vezes produz o mesmo resultado que uma. Quando o tratamento falha, a fila repete a tarefa um número limitado de vezes, com espera crescente (*backoff*). Falhas que não se resolvem repetindo, como um dado inválido, levam a tarefa ao conjunto de tarefas falhas, para análise (Taskforce.sh Inc., 2026). A Figura 7 resume o fluxo.

Figura 7: Fluxo de uma fila de mensagens com tentativas e tarefas falhas



Fonte: Autor.

O produtor insere a tarefa na fila (1) e o consumidor a retira e processa (2). Se falha e restam tentativas, ela volta à fila após a espera (3). Esgotadas as tentativas, vai para o conjunto de tarefas falhas (4).

Bibliotecas de filas para Node.js guardam o estado no Redis e oferecem repetição com *backoff*, tarefas periódicas, prioridades, limites de tarefas e remoção automática de tarefas concluídas (Taskforce.sh Inc., 2026). Como todo o estado fica no Redis, recomenda-se a configuração que recusa novas escritas, em vez de remover dados, quando a memória se esgota (Taskforce.sh Inc., 2026).

2.10 Banco de Dados

O banco de dados armazena, organiza e recupera informações de forma persistente. O modelo relacional estrutura os dados em tabelas com formato definido (esquema) e relações explícitas (Mohamed; Altrafi; Ismail, 2014), e a linguagem de consulta padrão é o SQL (*Structured Query Language*).

Bancos como o PostgreSQL organizam os dados em tabelas de linhas e colunas, com uma chave primária que identifica cada registro e chaves estrangeiras que ligam tabelas, garantindo a integridade referencial, isto é, que uma relação nunca aponte para um registro inexistente (PostgreSQL Global Development Group, 2026).

O particionamento divide uma tabela muito grande em partes menores, segundo um critério como o intervalo de tempo, com uma partição por mês, por exemplo. O banco ignora as partições que não podem conter os registros pedidos, técnica denominada poda de partições, e remover dados antigos descartando uma partição inteira é muito mais rápido que apagar milhões de linhas (PostgreSQL Global Development Group, 2026). É usual manter também tabelas de agregados, como médias por minuto ou por hora, que exibem longos períodos sem percorrer os registros brutos.

2.11 Redis

O *Redis* é um armazenamento de dados em memória, de código aberto, muito rápido, em que cada valor é encontrado por um nome, a chave. Ele mantém os dados em uso na memória RAM e responde rapidamente a leituras e escritas (Redis Ltd., 2026).

Ele pode gravar os dados em disco, apaga as chaves depois de um tempo de vida (*time-to-live* (TTL)) e troca mensagens entre processos por um mecanismo publicador-assinante (Pub/-Sub). É muito usado como cópia de acesso rápido (*cache*) e como canal de eventos (Redis Ltd., 2026).

O Redis oferece listas, conjuntos e outras estruturas, com operações atômicas, que acontecem de uma só vez, e executa pequenos programas em Lua sem interferência de outros clientes. Por isso serve para guardar o valor mais recente de uma informação, limitar pedidos por tempo, criar travas distribuídas e apoiar filas (Redis Ltd., 2026).

2.12 React e Next.js

O React é uma biblioteca JavaScript para montar interfaces a partir de componentes, pequenos blocos reutilizáveis de tela, como botões e gráficos. O desenvolvedor descreve como a interface deve ser para um dado estado, e o React atualiza apenas as partes da tela que mudaram (Meta Platforms, Inc., 2026).

O Next.js é um *framework* para aplicações web criado sobre o React. Sozinho, o React desenha a tela no navegador, depois que a página carrega, o que torna a primeira carga mais lenta (Vercel, Inc., 2026).

O Next.js oferece outras formas de gerar as páginas: no servidor, antes de enviá-las ao navegador, ou uma única vez, na publicação do sistema, para conteúdos que mudam pouco. A estratégia pode ser escolhida para cada tela (Vercel, Inc., 2026).

Ele também associa endereços a páginas pela organização dos arquivos do projeto e carrega apenas o código necessário a cada página (Vercel, Inc., 2026).

O *framework* distingue os componentes que executam no servidor dos que executam no navegador, necessários quando há interação (Vercel, Inc., 2026). Isso permite usar o Next.js só para as telas, enquanto as regras de funcionamento, a autenticação e o acesso aos dados ficam em um *back-end* separado, que o navegador consulta por pedidos HTTP.

2.13 Execução Isolada de Código

Às vezes um sistema precisa executar código escrito por seus usuários, como fórmulas e regras de cálculo. Fazer isso com as permissões da aplicação é arriscado, porque o programa

pode consumir todo o processador ou a memória, entrar em laço infinito, ler arquivos ou obter segredos. A execução isolada é um ambiente restrito em que o código roda com permissões e recursos limitados, por processos com poucas permissões, contêineres, máquinas virtuais ou interpretadores embutidos.

Uma técnica é executar o código em um interpretador pequeno, dentro de uma máquina virtual em que ele só acessa a própria memória, isolada da memória do programa que o executa, o hospedeiro. O código só alcança arquivos, rede ou relógio pelas funções que o hospedeiro liberar, e há limites de memória e de tempo, de modo que um laço infinito não trava o sistema. Criar um ambiente novo a cada execução evita que os dados de uma passem para a seguinte (Haas *et al.*, 2017).

2.14 Protocolo Modbus

O Modbus é um dos protocolos de comunicação mais difundidos na automação industrial. Ele funciona por pedido e resposta: o cliente, que em um sistema de supervisão é o supervisório, pede para ler ou escrever valores, e o dispositivo de campo, que atua como servidor, responde com os dados pedidos ou com um código de erro (Modbus Organization, 2006a).

Os dados de um dispositivo Modbus ficam em quatro áreas de memória. As bobinas e as entradas discretas guardam valores de um bit, como ligado ou desligado. Os registradores de retenção e os registradores de entrada guardam valores numéricos de 16 bits. As bobinas e os registradores de retenção aceitam leitura e escrita, e as entradas discretas e os registradores de entrada são somente de leitura. Cada operação tem um código de função, que indica a área de memória e se é uma leitura ou uma escrita (Modbus Organization, 2006a).

Para ler um valor, é preciso informar o equipamento, a área de memória e o endereço em que o dado está. Como essas informações são configuração, o mesmo programa se comunica com equipamentos de fabricantes diferentes sem mudar o código. Um valor que não cabe em 16 bits ocupa mais de um registrador, e a ordem em que eles são combinados varia entre fabricantes, por isso também precisa ser configurada. Quando necessário, uma escala converte o valor lido na grandeza real, como uma temperatura.

Como o cliente envia um pedido e espera a resposta antes do seguinte, o tempo de um ciclo de leitura é aproximadamente a soma dos tempos de cada pedido. Por isso, o valor mostrado ao operador pode ter até a idade de um ciclo completo, e ler registradores vizinhos em um único pedido reduz esse tempo.

3 Arquitetura e Requisitos do Sistema

Este capítulo especifica os requisitos da solução e apresenta a arquitetura que os atende: os requisitos funcionais e não funcionais, a arquitetura, a validação de cada pilar do SCADA e os serviços. O caminho de cada dado e de cada comando é tratado no capítulo seguinte.

3.1 Requisitos

Os requisitos funcionais descrevem o que a aplicação deve fazer, e os não funcionais estabelecem as qualidades e as restrições sob as quais ela opera. Boa parte deles deriva de sete conceitos de um sistema SCADA, aqui chamados de pilares: aquisição e registro de dados, sincronismo de tempo, níveis de acesso, comando e controle, escalabilidade, integrabilidade e redundância (Zanghi, 2019). Somam-se os próprios de uma solução orientada a serviços, como o cadastro das instalações, os valores calculados, a visualização e os alarmes.

3.1.1 Requisitos Funcionais

A Tabela 1 apresenta os requisitos funcionais do sistema, com a indicação do pilar do SCADA ou da funcionalidade complementar que os origina.

Tabela 1: Requisitos Funcionais.

Id	Requisito	Origem
RF01	Adquirir periodicamente os dados dos dispositivos de campo via Modbus, com período configurável por dispositivo	Aquisição
RF02	Registrar as leituras no histórico, com médias por minuto e por hora	Aquisição
RF03	Manter o valor atual de cada tag, com indicação de qualidade	Aquisição
RF04	Associar a cada leitura uma estampa de tempo	Sincronismo
RF05	Manter os usuários e controlar o acesso por cargo e por grupo de permissões	Níveis de acesso
RF06	Executar comandos nos dispositivos e validar o resultado pela resposta do equipamento	Comando
RF07	Executar escrita em hora marcada por evento agendado com script	Comando
RF08	Proteger o comando com trava por tag, idempotência e registro de auditoria	Comando
RF09	Incluir pontos, dispositivos e telas sem alterar o código	Escalabilidade
RF10	Manter o cadastro de plantas, dispositivos e tags, com pastas e mapeamento Modbus	Cadastro
RF11	Calcular valores derivados a partir de outras tags	Cálculo
RF12	Executar scripts do usuário em JavaScript de forma isolada	Cálculo
RF13	Exibir a lista de acompanhamento em tempo real e dashboards configuráveis	Visualização
RF14	Desenhar telas sinóticas e executá-las com valores e comandos	Visualização
RF15	Exibir as plantas em um mapa e consultar o histórico das tags	Visualização
RF16	Gerar alarmes em quatro níveis a partir de eventos, com reconhecimento	Alarmes

Id	Requisito	Origem
RF17	Exibir eventos, alarmes e comandos em uma linha do tempo	Alarmes

Fonte: Autor.

3.1.2 Requisitos Não Funcionais

A Tabela 2 apresenta os requisitos não funcionais, agrupados por categoria de atributo de qualidade: desempenho, escalabilidade, disponibilidade, segurança, interoperabilidade, manutenibilidade, portabilidade e usabilidade.

Tabela 2: Requisitos Não Funcionais.

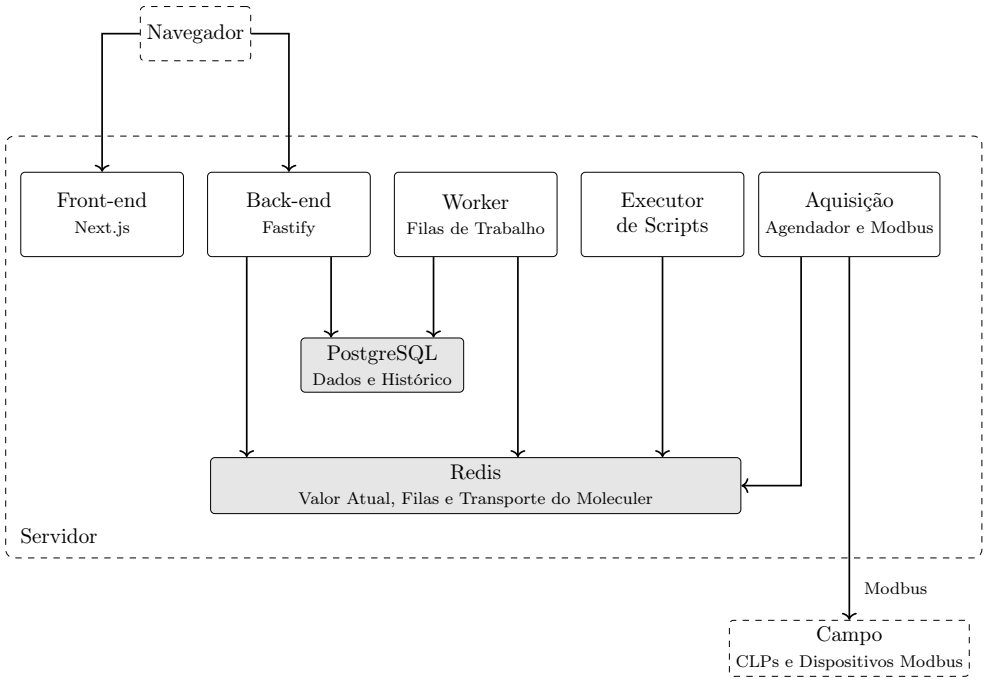
Id	Requisito	Categoria
RNF01	Atualizar as telas de tempo real com pouco atraso	Desempenho
RNF02	Permitir várias réplicas da aquisição e do processamento em segundo plano	Escalabilidade
RNF03	Continuar operando com funções reduzidas diante de falhas, sem mostrar dado incorreto	Disponibilidade
RNF04	Autenticar o acesso e autorizá-lo por cargo e por grupo	Segurança
RNF05	Proteger os comandos com travas e manter registro de auditoria	Segurança
RNF06	Executar o código do usuário sem rede, arquivos ou segredos e com limites de recursos	Segurança
RNF07	Comunicar com equipamentos de fabricantes distintos apenas por configuração Modbus	Interoperabilidade
RNF08	Dividir a solução em serviços que podem ser atualizados separadamente	Manutenibilidade
RNF09	Poder levar a solução a outro servidor sem alterar o código	Portabilidade
RNF10	Permitir o acesso por navegador, sem instalação local, com a interface em português	Usabilidade

Fonte: Autor.

3.2 Arquitetura

O sistema organiza-se em serviços, conforme a Figura 8. O navegador do operador chega ao servidor por duas vias: a das telas, atendida pelo *front-end*, e a dos dados e comandos, atendida pelo *back-end*. Os dispositivos de campo ficam fora do servidor e só o serviço de aquisição fala com eles.

Figura 8: Arquitetura do Sistema



Fonte: Autor.

São cinco serviços de aplicação e dois de dados. O *front-end*, em Next.js, entrega apenas as telas (Vercel, Inc., 2026). O *back-end*, em Fastify, concentra as regras de funcionamento, o cadastro, a autenticação e o acesso ao banco. O *worker* retira tarefas de filas e as executa em segundo plano, como a gravação do histórico, a avaliação de eventos e o cálculo de valores derivados. A aquisição reúne o agendador, que decide o instante de cada leitura, e o cliente Modbus, que fala com o dispositivo. O executor de scripts roda, de forma isolada, o JavaScript escrito pelos usuários. O PostgreSQL guarda o que precisa durar, e o Redis guarda o que precisa ser rápido.

Os serviços se comunicam pelo Moleculer, usando o Redis como transporte. O trabalho que pode ser feito depois e repetido em caso de falha passa por filas, também no Redis. Com isso, cada serviço fica pequeno e é possível criar réplicas sem que os demais precisem saber (Bigheti, 2020). A arquitetura segue o modelo orientado a serviços, discutido na literatura para aplicações industriais (Bigheti, 2020).

A aquisição e o executor de scripts não têm acesso ao banco nem às senhas dos armazenamentos: o primeiro recebe pronta a configuração de cada ponto e devolve o valor lido, e o segundo recebe o código e os valores e devolve o resultado. Isso reduz os pontos de ataque justamente onde o sistema lida com o mundo externo, o campo e o código do usuário (Stouffer *et al.*, 2015).

3.3 Validações dos Requisitos

A seguir, mostra-se como a arquitetura da Figura 8 atende a cada um dos pilares de um sistema SCADA (Zanghi, 2019).

3.3.1 Aquisição e Registro de Dados

As leituras (RF01 a RF03) são feitas pela aquisição, que conversa com os dispositivos de campo por Modbus (Modbus Organization, 2006a). Cada ponto é cadastrado como tag, com um tipo de dado (booleano, inteiro de 16 ou 32 bits, ponto flutuante, decimal codificado em binário ou texto) e a escala entre o valor bruto e a grandeza de engenharia. O período de varredura é definido por dispositivo, de 45 segundos por padrão. Redes seriais antigas são alcançadas por um *gateway*, sem exigir porta serial no servidor.

O valor atual de cada tag fica no Redis, com um indicador de qualidade (bom, desatualizado, falha, desconhecido ou implausível), e as telas o consultam sem sobrecarregar o banco. O histórico é gravado no PostgreSQL, em uma tabela dividida por mês, com médias por minuto e por hora que aceleram os gráficos longos (PostgreSQL Global Development Group, 2026). A gravação passa por filas, de modo que um pico de leituras não a derruba, e uma banda morta configurável evita registrar variações sem significado.

3.3.2 Sincronismo de Tempo

O sincronismo (RF04) vem do carimbo de tempo que a aquisição aplica ao ciclo de leitura. Todas as tags lidas no mesmo ciclo de um dispositivo recebem a mesma estampa, que indica o instante da amostragem e não o da transição do processo, como é próprio da leitura periódica. O carimbo acompanha o valor até a tela sem ser alterado, o que garante a cronologia para a análise de ocorrências (Zanghi, 2019). Como todos os serviços usam o relógio do mesmo servidor, a referência é comum, e com mais de um servidor o requisito passa a depender da sincronização entre as máquinas. Os instantes são guardados em UTC e convertidos para o fuso do usuário na exibição.

3.3.3 Níveis de Acesso

Nos níveis de acesso (RF05), há quatro cargos fixos: o administrador do sistema, que cuida da plataforma, o administrador da instalação, que configura o sistema, o supervisor, que cuida dos usuários e dos operadores, e o operador, que supervisiona e comanda. Há também grupos de permissões, criados a partir de um catálogo de dez permissões, que se somam quando a pessoa está em mais de um grupo. Assim se aplica o menor privilégio, e quem não pode comandar vê os valores, mas não escreve no equipamento (Stouffer *et al.*, 2015).

A cada pedido, o *back-end* verifica o cargo e as permissões do usuário, e uma alteração vale no pedido seguinte.

3.3.4 Comando e Controle

O comando e o controle (RF06 a RF08) seguem as variantes da literatura (Zanghi, 2019): o comando instantâneo, que muda o estado de um equipamento, e o controle por valor de referência, que define um alvo, ambos executados como escrita Modbus pela aquisição. O comando passa pela tela, que conduz o operador, pelo *back-end*, que verifica a permissão e guarda o registro, e pela aquisição, que escreve. Cada comando tem uma chave de idempotência,

identificador que impede a execução dupla, e uma trava por tag, que impede dois comandos simultâneos no mesmo ponto (Stouffer *et al.*, 2015).

O comando termina com a resposta do dispositivo, que já espelha o que foi aceito, sem releitura do ponto. Sem resposta no prazo, o comando falha, e quando o sistema perde o rastro, o estado é registrado como desconhecido, nunca como sucesso. Todo comando e todo reconhecimento de alarme ficam registrados com o autor e a origem. A escrita em hora marcada é feita por um evento agendado que dispara um script, com o nome do script registrado como origem.

3.3.5 Escalabilidade

A escalabilidade (RF09 e RNF02), ou seja, a capacidade de crescer, tem três frentes. Novos pontos, dispositivos e telas entram com o sistema em funcionamento, sem mexer no código. A aquisição e o *worker* podem rodar em várias réplicas, isto é, cópias do mesmo serviço trabalhando juntas, e cada dispositivo fica sempre com a mesma réplica, para que leituras e escritas não se atropelem.

Nos dados, o PostgreSQL cresce dividindo o histórico por mês, com retenção configurável, e o Redis tem um limite de memória e recusa novas escritas em vez de descartar dados em silêncio. Isso evita o limite de pontos comum nos supervisórios comerciais, embora o teto dependa da capacidade da máquina.

3.3.6 Integrabilidade

A integrabilidade (RNF07) vem do protocolo Modbus: a aquisição fala com dispositivos de fabricantes diferentes apenas por configuração, sem alterar o código. Dentro do servidor, os serviços trocam dados pelo Moleculer e pelas filas, e o navegador fala com o *back-end* por HTTP, com dados em JSON. A troca de dados com outros programas fica como evolução.

3.3.7 Redundância

A redundância (RNF03) apoia-se nas réplicas e nas filas, que sobrevivem a falhas (Zanghi, 2019). Cada réplica da aquisição e do *worker* retira trabalho das mesmas filas, de modo que a perda de uma é absorvida pelas demais, e o *front-end* e o *back-end* também rodam em duas réplicas. Os trabalhos que falham são repetidos com espera controlada, os que esgotam as tentativas ficam guardados para inspeção, e o histórico tolera pausas curtas do banco. As travas de comando têm prazo de validade, de modo que a queda de uma cópia nunca as deixa presas.

O ponto mais sensível é o Redis, que concentra o valor atual, as filas e o transporte entre serviços, e por isso seu conteúdo pode ser recuperado. Quando ele volta vazio, a ausência de um marcador, o *boot-marker*, aciona uma rotina que recoloca o valor de cada tag a partir da última leitura no banco, agenda de novo as tarefas periódicas e refaz o estado dos alarmes. Enquanto isso, as funções que dependem dele respondem que estão indisponíveis, e os valores recolocados ficam marcados como desatualizados até a próxima leitura.

A replicação do PostgreSQL com comutação automática e a execução em mais de uma máquina ficam como evolução. Como os serviços não guardam dados importantes dentro de si, isso não exige mudança no código, apenas na forma de instalá-los.

3.4 Serviços

Cada serviço pode ser iniciado e parado separadamente, segundo o modelo orientado a serviços (Bigheti, 2020), o que permite aumentar a capacidade só onde é preciso e impede que a falha de um derrube os outros. A Tabela 3 resume a função de cada serviço.

Tabela 3: Serviços e responsabilidades.

Serviço	**Responsabilidade**
Front-end	Entregar as telas ao navegador, sem regra de funcionamento
Back-end	Atender os pedidos, autenticar, autorizar, cadastrar, consultar e colocar trabalho nas filas
Worker	Executar as tarefas das filas: histórico, eventos e cálculos
Aquisição	Agendar e executar a leitura e a escrita Modbus nos dispositivos de campo
Executor de scripts	Executar o JavaScript do usuário de forma isolada
PostgreSQL	Guardar de forma permanente o cadastro, o histórico e os demais dados
Redis	Guardar, para busca rápida, o valor atual, as filas e as travas, e transportar as mensagens do Moleculer

Fonte: Autor.

3.4.1 Front-end

O *front-end* é desenvolvido em Next.js com React e entrega apenas as telas (Vercel, Inc., 2026), sem banco de dados e sem regras de funcionamento. Depois de carregadas as telas, o navegador chama o *back-end* diretamente.

As telas de tempo real funcionam por consultas periódicas, e uma falha gera aviso de conexão e nova tentativa. A interface usa um conjunto próprio de componentes, com tema claro e textos em português.

3.4.2 Back-end

O *back-end* é o centro do sistema, desenvolvido em Node.js com o *framework* Fastify. Antes de atender um pedido, o *back-end* confere o formato dos dados, descrito em um esquema compartilhado com a interface. Ele faz o *login* e a verificação de permissões, o cadastro, as consultas ao histórico, a gestão de usuários, a edição de telas, scripts e eventos e o envio dos comandos.

Além do *worker*, o *back-end* é o único serviço que fala com o banco. Ele usa o Redis para ler o valor atual e controlar as travas, e o Moleculer para chamar a aquisição e o executor. Nenhuma tarefa de fundo roda junto dele, para que um trabalho pesado não atrase as telas: o que precisa ser executado depois ou repetido vai para uma fila, a cargo do *worker*.

3.4.3 Worker

O *worker*, em TypeScript, não atende pedidos do navegador: apenas processa as filas de trabalho. Ele grava o histórico, atualiza o valor atual e as médias, avalia os eventos, dispara os agendados, executa as rotinas de manutenção e calcula as tags internas e as variáveis meta,

pedindo a avaliação de scripts ao executor. O *worker* não fala com o campo, e sua capacidade pode ser aumentada com réplicas.

3.4.4 Aquisição

A aquisição é a fronteira com o campo e o único componente que conversa em Modbus. Ela roda em Node, sem banco de dados, e reúne duas funções. O agendador decide o instante de cada leitura, agrupa as tags de um dispositivo em uma rodada para reduzir o tráfego e não inicia um ciclo enquanto o anterior não terminar. O cliente Modbus monta o quadro, valida a resposta e os códigos de erro, decodifica os registradores, aplicando a ordem de bytes e a escala, e produz o valor na unidade de engenharia (Modbus Organization, 2006a).

Ela recebe pronta a configuração de cada dispositivo e pode rodar em réplicas, com cada dispositivo fixo em uma delas. A trava por dispositivo, no Redis, dá ao comando prioridade sobre a leitura periódica. As falhas de comunicação são contadas por dispositivo, com tempo limite e número de tentativas configuráveis. Quando uma leitura falha, o último valor é mantido e marcado como desatualizado, e a ocorrência alimenta os eventos do dispositivo, pois, em um SCADA, a ausência de um dado precisa ser tão visível quanto um limite ultrapassado.

Os resultados saem como mensagens, que o *worker* transforma em histórico, valor atual e eventos, e as escritas de comando chegam pelo Moleculer. A aquisição não guarda dados, não decide permissões e não executa scripts, o que facilita replicá-la, testá-la e acrescentar outros protocolos industriais sem alterar o restante.

3.4.5 Executor de Scripts

O executor roda o JavaScript dos usuários, presente na variável meta, no script de um evento, no componente de conteúdo do sinótico e na tag interna gerada. Deixar o usuário escrever código em um sistema industrial exige isolamento forte. Por isso, cada avaliação roda em uma cópia nova de um interpretador, em um ambiente fechado, em um processo hospedeiro que é reciclado. O código não acessa rede, arquivos nem segredos, e cada execução tem limites de tempo, memória, recursão e saída.

O serviço não tem credencial de banco: o *back-end* ou o *worker* leem o banco e enviam os valores no pedido. A conta dele no Redis tem o mínimo de permissões, e um limite global de execuções simultâneas impede que um script lento ocupe o serviço inteiro. Assim, mesmo que um script malicioso escape do interpretador, não alcança o banco, a rede interna nem os comandos de escrita do campo.

3.4.6 PostgreSQL

O PostgreSQL é um banco relacional de código aberto, com bom suporte a particionamento e a consultas em janelas de tempo (PostgreSQL Global Development Group, 2026), e guarda os usuários, as sessões, o cadastro, o histórico, os alarmes, as telas, os scripts e os grupos de permissões.

As conexões são separadas por finalidade, de modo que a gravação de leituras, as telas e as consultas pesadas não as disputam. As mudanças na estrutura do banco são feitas por migrações, roteiros de alteração compatíveis com a versão anterior do código e ensaiados antes da aplicação. O histórico é dividido por mês, com retenção configurável, e cada consulta nova nasce com o índice que a atende.

3.4.7 Redis

O Redis é um armazenamento de dados em memória (Redis Ltd., 2026) com várias funções. Como memória de acesso rápido (*cache*), mantém o valor atual de cada tag. Guarda os trabalhos das filas, transporta a comunicação do Moleculer e guarda os contadores de limites, as travas distribuídas e a lista de sessões revogadas.

Os contadores e as travas têm prazo de validade. Já o último valor de cada tag não expira, e por isso o Redis é reconstruído do banco quando volta vazio. Ele só é acessível pela rede interna, e cada serviço tem uma conta com permissões próprias.

3.4.8 Comunicação Moleculer e Filas

O Moleculer usa o Redis como transporte (MoleculerJS, 2026) e é por ele que o *back-end* pede uma escrita à aquisição, que a aquisição entrega os resultados das leituras e que o *back-end* e o *worker* pedem a avaliação de scripts. Ele encontra os serviços automaticamente, distribui as chamadas entre as réplicas, aplica tempo limite e impede que uma falha se propague.

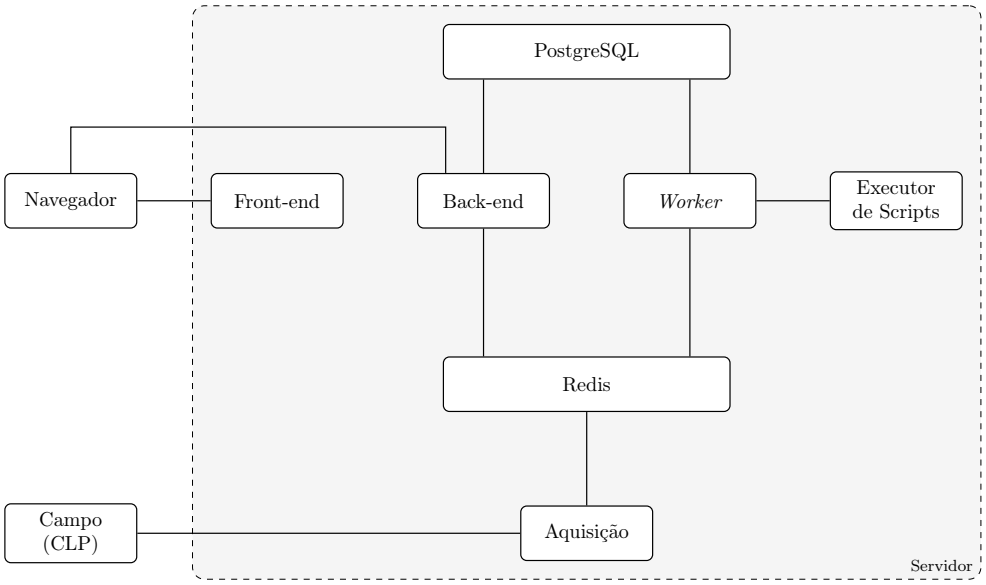
O trabalho sem resposta imediata passa por filas, que limitam as tarefas simultâneas, descartam os trabalhos de histórico excedentes quando a memória do Redis passa de um limiar e guardam por um tempo os que falharam. Cada tarefa deve ser idempotente, isto é, repeti-la não produz efeito duplicado. O Moleculer cobre a chamada que espera resposta, e as filas cobrem o trabalho que precisa sobreviver a falhas.

4 Fluxos do Sistema

Este capítulo descreve os caminhos que o dado e as ações percorrem no sistema: quem pede, quem responde, onde cada informação é guardada e o que acontece quando uma peça falha. Cada fluxo é apresentado por um diagrama de blocos com setas numeradas, seguido de um texto que percorre os passos na ordem.

Os blocos correspondem aos serviços da arquitetura e têm as posições fixas da Figura 9. O navegador e o equipamento de campo ficam fora do servidor. Dentro dele estão o *front-end*, que entrega as telas, o *back-end*, que concentra as regras de funcionamento, o *worker*, que retira tarefas das filas e grava no banco, a aquisição, que conversa com o equipamento, e o executor de scripts, que roda, de forma isolada, o JavaScript escrito pelos usuários. O PostgreSQL guarda os dados que precisam durar, e o Redis guarda o valor atual de cada tag, as filas e as travas e transporta a comunicação entre os serviços.

Figura 9: Mapa dos blocos usados nos fluxos



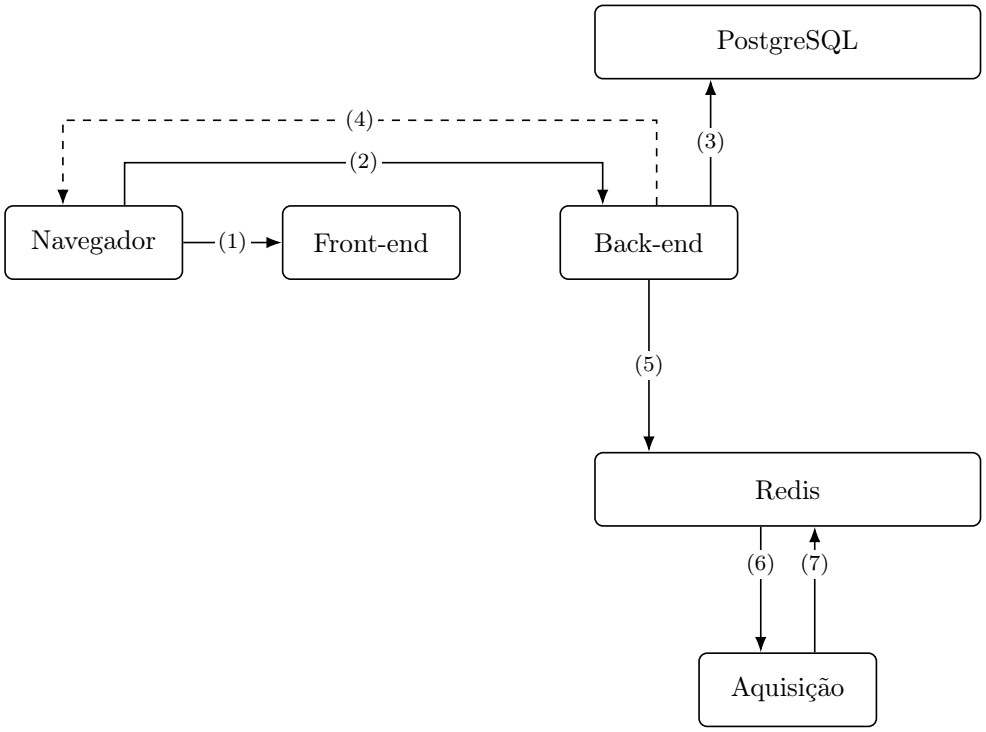
Fonte: Autor.

Em todas as figuras, a linha cheia indica o caminho normal, a tracejada indica uma resposta e a de traço e ponto indica um caminho alternativo, percorrido quando alguma peça falha. Os números entre parênteses seguem a ordem dos passos, e o mesmo número em dois trechos de linha indica um único passo que atravessa o Redis, como nas chamadas entre o *back-end* e a aquisição, que o Moleculer transporta por esse banco (MoleculerJS, 2026; Redis Ltd., 2026). Os passos alternativos recebem o número do passo normal e uma letra, como em (4b). Cada fluxo termina com o que acontece quando uma peça deixa de funcionar.

4.1 Cadastro de uma Tag

O cadastro de uma tag é o ponto em que o sistema passa a conhecer uma nova variável do processo. A tag pertence a um dispositivo de entrada e saída, ligado a uma planta, e traz o mapeamento do ponto no equipamento, a escala de engenharia e as regras de histórico. O cadastro é feito com o sistema em funcionamento, e a nova tag chega à aquisição sem reiniciar nenhum processo, como mostra a Figura 10.

Figura 10: Fluxo do cadastro de uma tag



Fonte: Autor.

O administrador abre a tela de tags de um dispositivo, entregue pelo *front-end* (1). Ao salvar o formulário, o navegador envia o pedido ao *back-end* (2), que confere a permissão do usuário e as regras do cadastro e grava a tag e o registro da mudança no banco (3).

O *back-end* responde ao navegador (4) e publica o evento de mudança do cadastro (5), que o Moleculer entrega, pelo Redis, ao Agendador, dentro da aquisição (6). Como a aquisição não tem banco de dados, o Agendador pede ao *back-end* a configuração pronta do dispositivo (7) e inclui a tag no plano de leitura. A primeira leitura acontece no ciclo seguinte, e até lá a tela mostra a tag como sem dado.

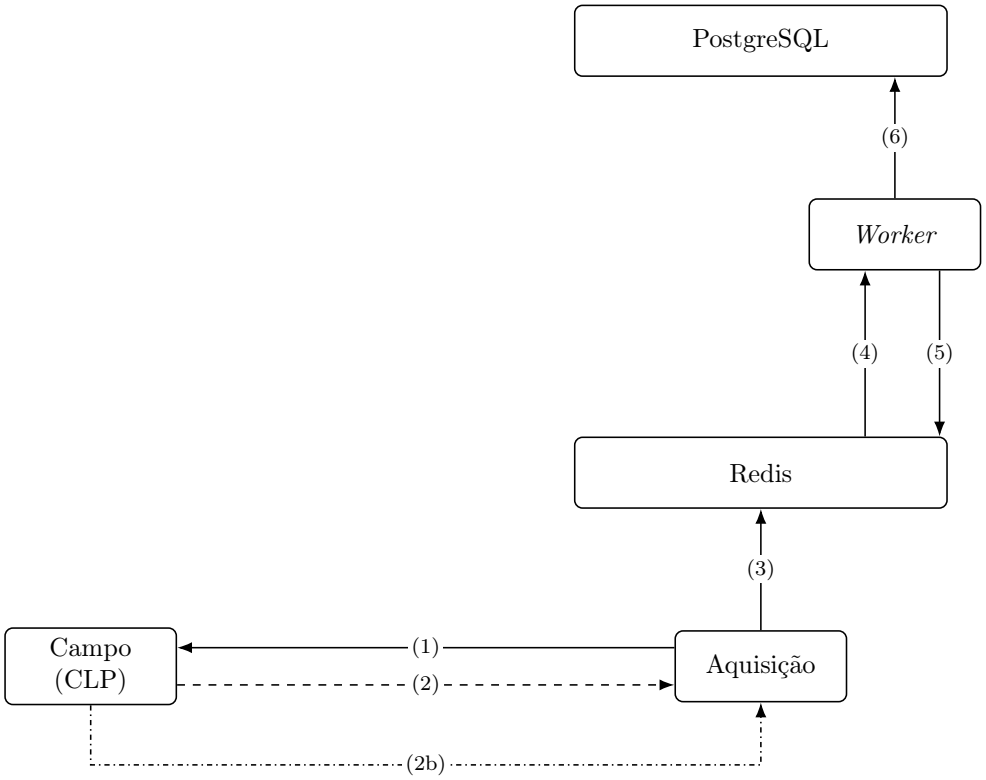
Quando a validação falha, o *back-end* recusa o pedido com uma mensagem, e nada é gravado. Se o processo cai entre a gravação e a publicação do evento, a mudança continua registrada e é publicada depois. A cada cinco minutos, o Agendador compara as versões do cadastro de cada planta e relê as que mudaram.

Sem o *back-end*, o Redis ou o banco, o cadastro não é possível, mas a leitura do campo continua, porque a aquisição mantém a última configuração recebida. Com a aquisição parada, o cadastro funciona normalmente, e ao voltar ela atualiza as plantas que mudaram.

4.2 Do Campo ao Banco

Este é o fluxo principal de um supervisório, da leitura no equipamento até o histórico no banco, como mostra a Figura 11.

Figura 11: Fluxo da leitura do equipamento até o banco



Fonte: Autor.

No período de cada dispositivo, a aquisição envia uma requisição Modbus ao CLP (1) (Modbus Organization, 2006a), que responde com os valores brutos (2). A aquisição registra o instante da resposta, aplica a escala de engenharia, atribui a qualidade e publica o resultado em duas filas no Redis, uma de valor atual, chamada de ingestão, e outra de histórico (3).

O *worker* retira as leituras das filas (4), grava o valor atual no Redis (5) e grava o histórico no banco (6), conforme o modo da tag: não gravar, gravar toda leitura ou gravar quando o valor muda além da banda morta. Tarefas periódicas calculam no banco as médias por minuto e por hora, usadas nas consultas de períodos longos.

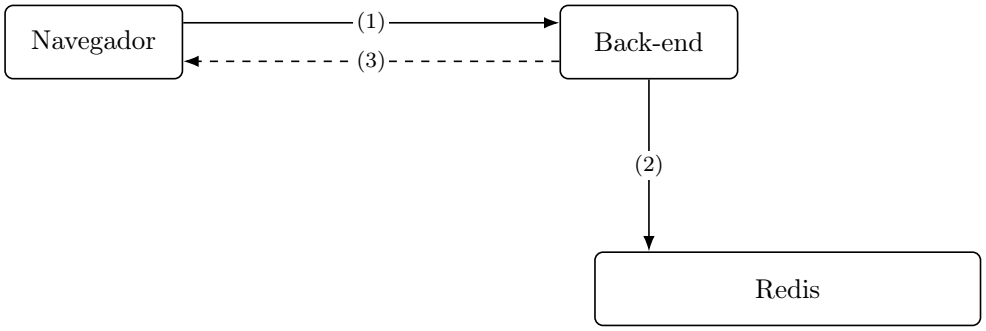
Se o equipamento não responde ou devolve um erro (2b), a aquisição publica as tags com qualidade ruim, e o *worker* mantém o último valor, com a indicação da falha. Depois de três ciclos falhos seguidos, por padrão, o evento de falha de comunicação do dispositivo é ativado, e o primeiro ciclo bem-sucedido o encerra.

Se o *worker* para, as leituras se acumulam nas filas e são processadas quando ele volta. Se o Redis volta vazio, uma rotina refaz o valor atual a partir do banco. Se o banco fica indisponível, os trabalhos de histórico esperam e tentam de novo.

4.3 Do Banco à Tela em Tempo Real

A tela obtém o valor atual por consultas periódicas: ela pergunta os valores das tags da planta aberta, e a resposta vem apenas do Redis, sem consultar o banco nem o equipamento, como resume a Figura 12.

Figura 12: Fluxo da exibição do valor atual por consulta periódica



Fonte: Autor.

A tela envia o pedido ao *back-end* (1), que lê do Redis o valor atual de cada tag (2) e define sua qualidade. Se nada mudou, a resposta é vazia. Caso contrário, o *back-end* devolve os valores (3), e a tela atualiza o que mudou.

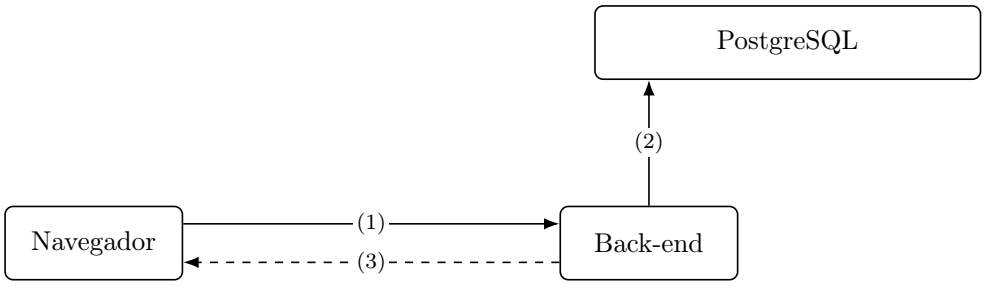
A qualidade aparece com texto e ícone. O valor parado há mais de três períodos aparece como desatualizado, e a tag cuja leitura falha mostra o último valor bom com um símbolo de alerta e o texto de falha de comunicação. A lista de acompanhamento, o sinótico, o medidor do painel e a grade de tags usam a mesma consulta e a mesma forma de exibição. Depois de três falhas seguidas de pedido, a tela mostra o aviso de conexão instável e aumenta o intervalo entre as tentativas.

Com o Redis fora do ar ou vazio, ou com o banco fora do ar, a tela mostra que o serviço está indisponível em vez de valores. A queda do equipamento de campo não derruba a tela, que continua a responder com o último valor e a qualidade correspondente.

4.4 Histórico e Tendência

A consulta ao histórico alimenta os gráficos de tendência. O sistema escolhe, pelo tamanho da janela, entre a leitura bruta e os resumos já calculados, como mostra a Figura 13.

Figura 13: Fluxo da consulta de tendência



Fonte: Autor.

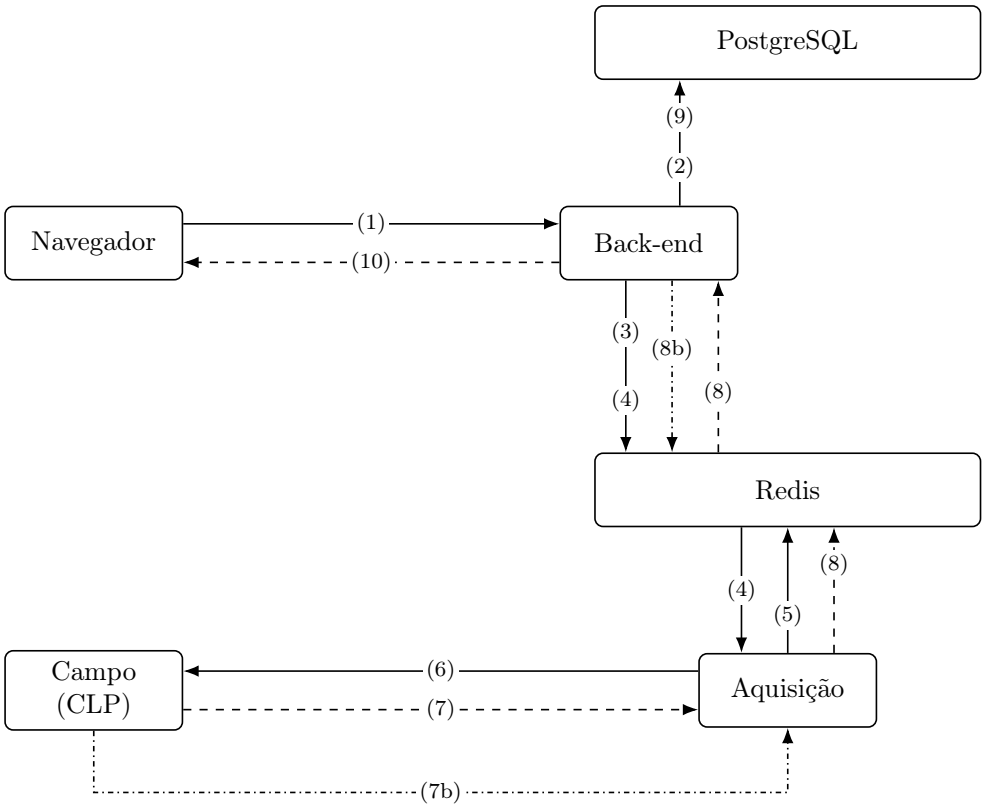
A tela envia o pedido de tendência (1), com as tags e o intervalo desejado. O detalhe depende da janela: até uma hora, as leituras brutas, até um dia, a média de um minuto, até sete dias, a de dez minutos, até trinta e um dias, a de trinta minutos, e acima disso a de seis horas. Toda média vem com o mínimo e o máximo do intervalo, para que um pico curto não desapareça.

O *back-end* consulta o banco (2), lendo os resumos e completando o início e o fim do intervalo com as leituras brutas, e devolve o resultado (3). A tela desenha então a série. Depois da primeira carga, a tela pede apenas o trecho novo. Um pedido grande demais é recusado, e a falha da consulta não prejudica a exibição dos valores atuais, que têm caminho próprio.

4.5 Comando de Equipamentos

O comando é a escrita de um valor em um ponto do equipamento, como ligar um motor ou alterar um ponto de ajuste. Na tela, o operador escolhe a tag, informa o novo valor e confirma, e o navegador envia o pedido ao *back-end* (1), como mostra a Figura 14.

Figura 14: Fluxo do comando



Fonte: Autor.

O *back-end* confere a permissão do usuário, se a tag permite escrita, se o dispositivo se comunica e se o valor está dentro da faixa, e registra o comando no banco (2). Sem a certeza de que o equipamento se comunica, o comando é recusado. Um pedido repetido não cria um segundo comando, e cada tag tem no máximo um comando em andamento.

O *back-end* obtém no Redis a trava da tag (3) e chama a ação de escrita da aquisição pelo Moleculer (4), sem fila e sem nova tentativa automática. A aquisição obtém no Redis a trava do dispositivo (5), pois o comando tem prioridade sobre a leitura periódica, e envia a escrita ao CLP (6). O sucesso é a resposta normal do equipamento, que repete o pedido enviado (7) e é conferida contra o endereço e o valor, e a próxima leitura periódica mostra o efeito real. A aquisição devolve o resultado ao *back-end* (8), que fecha o comando, grava o registro (9) e informa o diálogo (10). O registro guarda quem emitiu o comando, o valor, o horário e o resultado, e não pode ser apagado nem alterado.

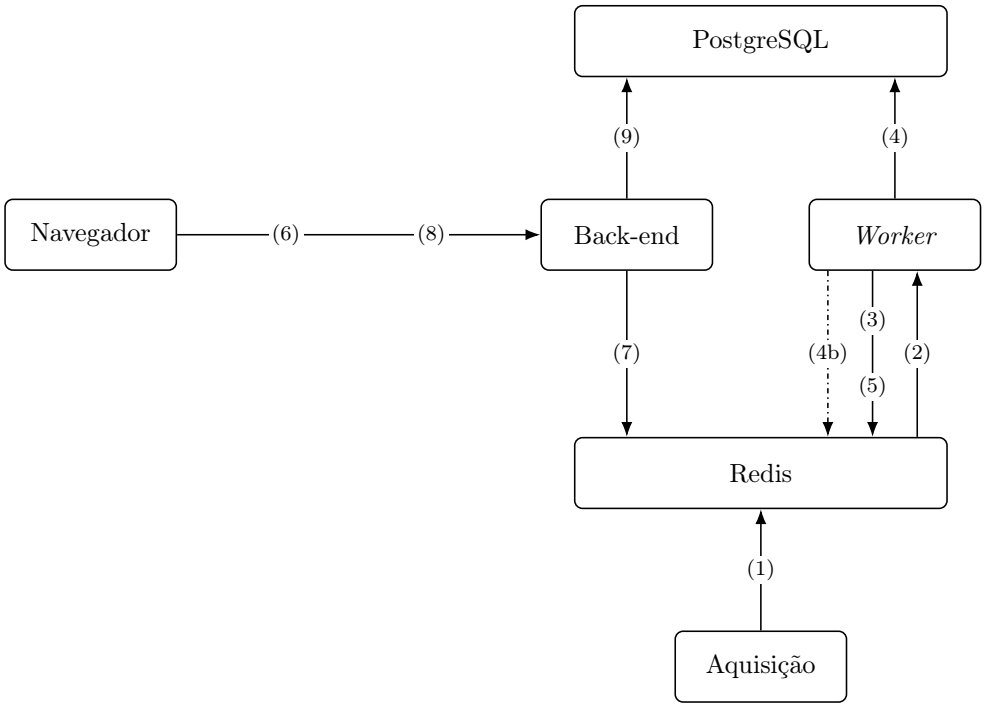
Além do usuário, um script também pode escrever, inclusive em hora marcada, por um evento agendado. Essa escrita usa o mesmo caminho do comando e cede ao comando do operador. A tag interna, que não tem equipamento, recebe o valor diretamente, sem passar pela aquisição.

Se o equipamento responde com exceção, com uma repetição que não confere ou não responde no prazo (7b), a escrita é tratada como falha, e a mensagem pede que o operador confira o valor do ponto. Se o sistema perde o rastro, por exemplo, se a resposta da aquisição não chega ao *back-end* (8b), o comando é fechado como sem confirmação do equipamento, e a tag fica bloqueada por trinta e cinco segundos. Quando uma peça falha, o sistema prefere recusar a arriscar: sem o Redis, o Agendador ou a aquisição, o comando é recusado.

4.6 Alarme

O alarme transforma uma condição do processo em um aviso que exige a atenção de uma pessoa. A condição é um evento da tag: a tag, um operador de comparação, o valor de referência e uma banda morta opcional, que evita que um valor oscilando em torno do limite abra e feche o alarme. O alarme liga um ou mais eventos a um de quatro níveis fixos, Crítico, Alto, Médio e Baixo, como mostra a Figura 15.

Figura 15: Fluxo da detecção, da sinalização e do reconhecimento de um alarme



Fonte: Autor.

A leitura é publicada pela aquisição na fila de ingestão (1) e retirada pelo *worker* (2), que, depois de gravar o valor atual, avalia os eventos da tag. A avaliação é feita no Redis, que guarda o estado de cada detector (3): compara o valor com o limite, aplica a banda morta e decide se houve transição. Havendo transição, o *worker* a grava no banco (4), abrindo, atualizando ou fechando a ocorrência do alarme, e atualiza no Redis o sumário de alarmes (5).

A moldura da aplicação, presente em todas as páginas, pergunta periodicamente o sumário de alarmes (6), que o *back-end* lê no Redis (7). O indicador exibe o nível mais grave não

reconhecido e a quantidade de ocorrências. O operador reconhece uma ocorrência na página de alarmes (8), e o *back-end* grava o reconhecimento, com o nome da pessoa (9).

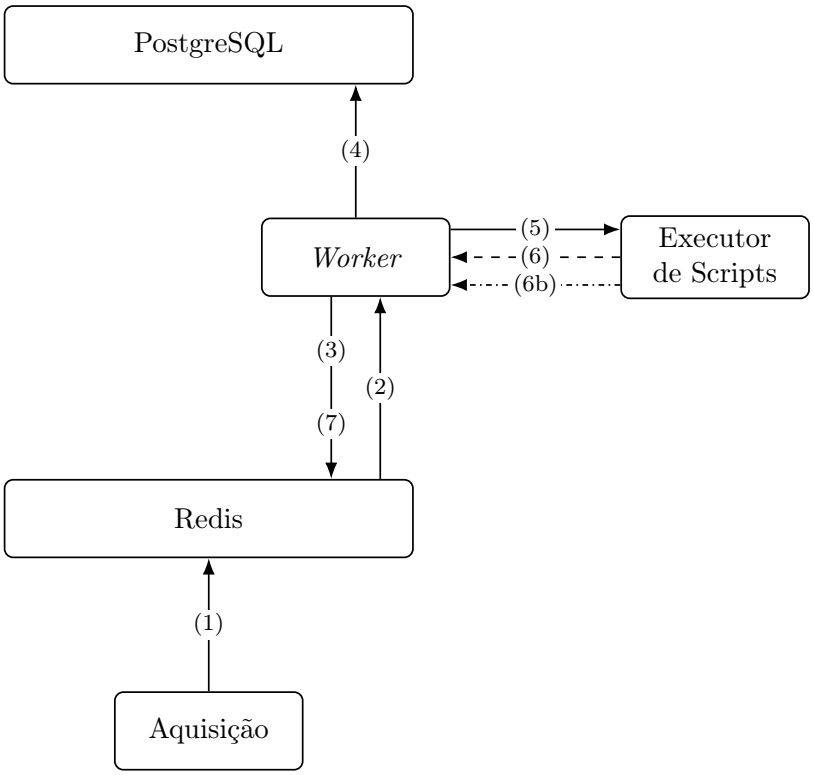
O evento composto combina outros eventos com os operadores E, OU e NÃO, e o evento agendado dispara em horários definidos. Os eventos de falha de comunicação, de leitura e de escrita são criados automaticamente com o dispositivo Modbus. Se o processo cai antes da gravação no banco (4b), a transição não se perde: uma tarefa periódica grava o que ficou pendente no Redis.

Com o Redis fora, a avaliação para, e o trabalho é repetido. Com o banco indisponível, as transições ficam pendentes até ele voltar. Com o *worker* parado, nenhum alarme novo é detectado, e a fila preserva as leituras.

4.7 Variável Meta

A variável meta é uma tag cujo valor vem de um cálculo escrito pelo administrador sobre outras tags da mesma planta, como uma diferença de pressões. O código, em JavaScript, roda no executor de scripts, isolado, sem banco, rede, arquivo nem segredo, como mostra a Figura 16.

Figura 16: Fluxo do cálculo de uma variável meta



Fonte: Autor.

A variável é recalculada a cada leitura nova de uma tag que o código lê ou em horários definidos. A aquisição publica a leitura na fila de ingestão (1), e o *worker* a retira (2), identifica quais variáveis meta leem aquela tag e coloca os cálculos em uma fila própria (3).

O *worker* retira os cálculos e busca os dados de que o código precisa: o valor atual, no Redis, e os pontos de histórico, no banco (4). Os dados seguem com o código ao executor de scripts, pelo Moleculer (5). Para cada cálculo, o executor cria uma cópia nova do interpretador, com limites de tempo, memória e saída, e devolve o valor ou o erro (6).

O *worker* valida o que veio e o publica como leitura da tag interna (7), pelo mesmo caminho das leituras do equipamento. A qualidade da meta é a pior entre as tags que ela lê. O mesmo executor roda a tag interna gerada, o script de evento e o componente de conteúdo do sinótico. Se o código ultrapassa um limite ou gera um erro (6b), apenas aquela meta recebe erro, e o *worker* publica a qualidade ruim em vez de um valor.

Se o executor cai, nenhuma meta é calculada, e a variável fica com o último valor. As demais tags, seus alarmes e os comandos não dependem dele.

4.8 Efeito das Falhas

A Tabela 4 reúne o efeito da falha de cada bloco e mostra que o sistema perde apenas as funções ligadas à peça que falhou.

Tabela 4: Efeito da falha de cada bloco nos fluxos do sistema.

Bloco que falha	**O que deixa de funcionar e o que continua**
Front-end	Páginas novas deixam de ser entregues. As telas já abertas continuam a consultar o *back-end*, e a aquisição, o histórico e os alarmes não são afetados.
Back-end	Cadastro, login, comando e consultas deixam de responder. A aquisição continua a ler com a última configuração, e o *worker* continua a gravar o valor atual, o histórico e a avaliar os alarmes.
Worker	O valor atual, o histórico, os alarmes e as variáveis meta deixam de avançar, e as leituras se acumulam nas filas. As telas continuam a responder com o último valor, que passa a desatualizado.
Aquisição	Não há leitura nem escrita no equipamento, e os valores passam a desatualizados. O cadastro, as telas e os alarmes já abertos continuam disponíveis, e o comando é recusado.
Executor de scripts	Variáveis meta, scripts e componentes de conteúdo param, e a variável meta fica com o último valor. As tags de equipamento, os alarmes sobre elas e os comandos seguem normalmente.
Redis	As funções que exigem login e o próprio login respondem indisponibilidade, a aquisição não publica e o comando não executa. Ao voltar vazio, a rotina de reconstrução refaz o valor atual, os estados e a lista de sessões revogadas.
PostgreSQL	Cadastro, login, histórico e alarmes deixam de funcionar, e a tela de valores informa indisponibilidade. O valor atual continua a ser atualizado no Redis.
Campo	O último valor é mantido e passa a desatualizado, o evento de falha de comunicação do dispositivo é ativado e o comando é recusado. Os demais dispositivos não são afetados.

Fonte: Autor.

4.9 Resumo dos Fluxos

A Tabela 5 reúne os sete fluxos, os blocos que cada um percorre e o que transporta o trabalho entre eles. A comunicação entre os serviços usa dois mecanismos: a chamada do Moleculer, quando o solicitante precisa da resposta, como no comando, e a fila, quando o trabalho pode esperar e não pode ser perdido, como no histórico. Entre o navegador e o *back-end*, o transporte é o HTTP com dados em JSON.

Tabela 5: Resumo dos fluxos do sistema.

Fluxo	**Blocos envolvidos**	**Fila ou ação usada**
Cadastro de uma tag	Navegador, Front-end, Back-end, Redis, PostgreSQL, Aquisição	HTTP, evento de mudança do cadastro e ações de registro do Moleculer
Do campo ao banco	Aquisição, Campo, Redis, *Worker*, PostgreSQL	Ação de leitura do Moleculer, filas de ingestão e de histórico, fila geral de tarefas periódicas
Valor atual em tempo real	Navegador, Back-end, Redis	HTTP com consulta periódica, leitura direta do Redis
Histórico e tendência	Navegador, Back-end, PostgreSQL	HTTP, consulta somente leitura
Comando de equipamentos	Navegador, Back-end, Redis, PostgreSQL, Aquisição, Campo	HTTP, ação de escrita do Moleculer, evento de escrita concluída
Alarme	Aquisição, Redis, *Worker*, PostgreSQL, Back-end, Navegador	Fila de ingestão, fila geral de tarefas, sumário no Redis
Variável meta	Aquisição, Redis, *Worker*, Executor de scripts, PostgreSQL	Fila de ingestão, fila de ondas de cálculo, ação de avaliação do Moleculer

Fonte: Autor.

O que precisa de resposta imediata passa pelo *back-end*, que sempre confere a permissão. O que não pode se perder passa por uma fila e é processado por um *worker*. O valor atual fica no Redis e o histórico no PostgreSQL, de modo que a tela em tempo real nunca consulta o banco. Em cada fluxo, a falha de uma peça leva à recusa do pedido, à repetição do trabalho ou à marcação do valor como desatualizado, sem produzir um dado falso.

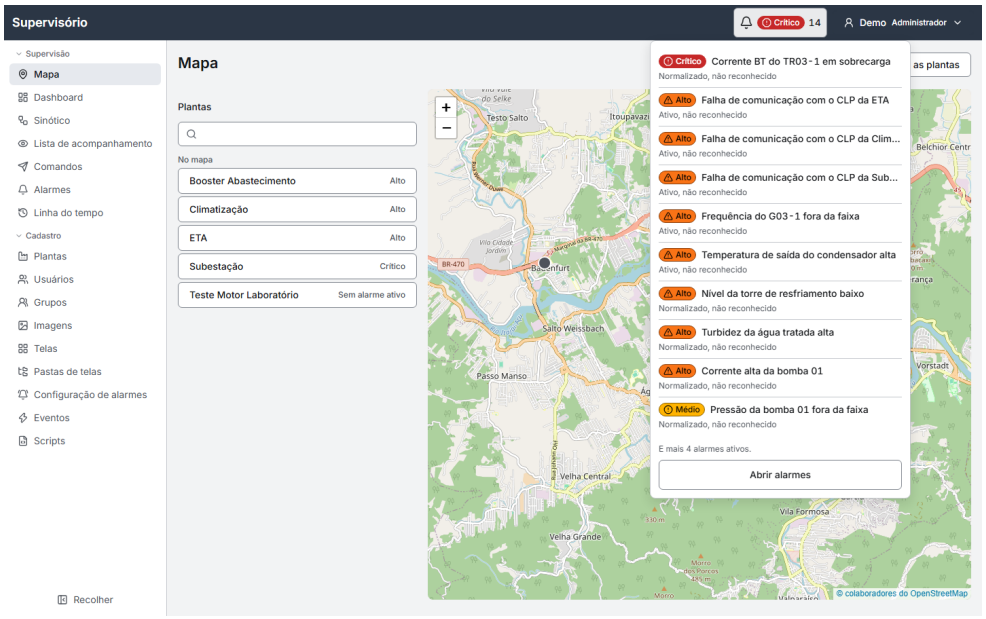
5 Interface do Sistema

Este capítulo descreve a interface como o operador e o administrador a utilizam: a estrutura geral da aplicação e cada grupo de telas, com o requisito que atende. Os nomes de menu e de campos são os que aparecem na interface.

5.1 Estrutura geral da aplicação

Toda tela depois do acesso fica dentro de uma moldura comum, mostrada na Figura 17. À esquerda, o menu lateral lista apenas o que o cargo e as permissões permitem abrir (RF05). Na barra superior ficam o indicador de alarmes e o menu do usuário. O acesso é pelo navegador, sem instalação local (RNF10).

Figura 17: Casco da aplicação com menu lateral e indicador de alarmes



Fonte: Autor.

O menu segue a hierarquia de telas: primeiro os itens de supervisão, Mapa, Dashboard, Sinótico, Lista de acompanhamento, Comandos, Alarmes e Linha do tempo, e depois os de configuração, exclusivos do administrador da instalação: Plantas, Usuários, Grupos, Imagens, Telas, Pastas de telas, Configuração de alarmes, Eventos e Scripts. Usuários é visível também ao supervisor, que gere somente operadores.

O indicador de alarmes mostra um sino com o nível mais grave não reconhecido e a quantidade (RF16). Ele pulsa apenas com alarme Crítico ou Alto não reconhecido. Se os dados do indicador estiverem velhos por falha de rede, um relógio aparece junto ao sino.

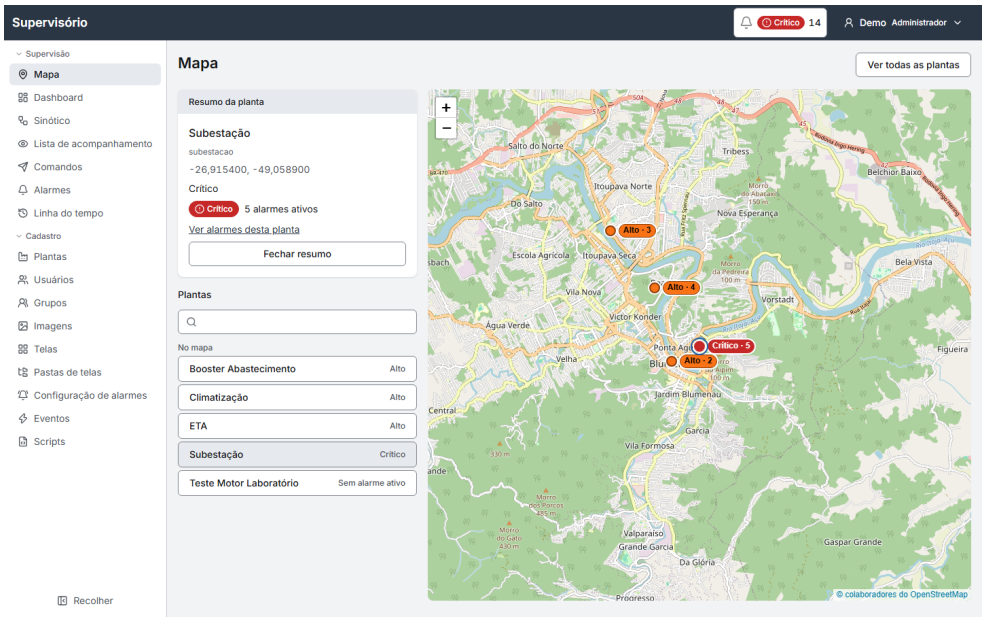
As telas com valores ao vivo exibem uma faixa de estado da conexão, que avisa quando a rede oscila ou o serviço não responde, e o último valor continua na tela, marcado como desatualizado. Os valores são atualizados por consultas periódicas (RNF01).

5.2 Mapa das plantas

O Mapa é a visão geral do sistema, com um pino por planta com coordenada, como mostra a Figura 18. O pino de uma planta sem alarme é neutro, e o de uma planta com alarme assume

a cor do mais grave e traz o nome do nível. Uma lista lateral repete todas as plantas. O clique abre um resumo com a quantidade de alarmes e o atalho “Ver alarmes desta planta”.

Figura 18: Mapa das plantas com o estado de alarme de cada uma



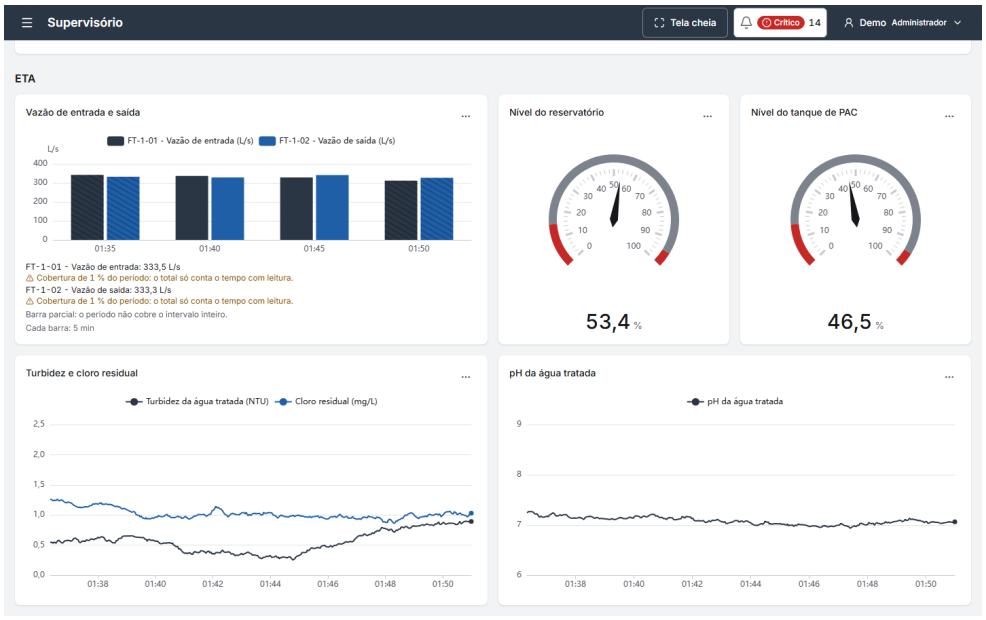
Fonte: Autor.

O mapa de fundo vem de um serviço de terceiros, e por isso uma nota fixa informa o que é enviado a ele, e uma configuração abre o mapa sem fundo, sem chamada externa. Quem não pode ver alarmes enxerga pinos neutros, e o servidor nem envia esses dados (RF15).

5.3 Dashboard

No Dashboard, o administrador monta painéis de tendência e de situação com gráficos, e o operador apenas observa, como mostra a Figura 19.

Figura 19: Dashboard em execução com painéis de linha, barra e medidor



Fonte: Autor.

O painel de linha mostra até 12 tags. Até uma hora cada ponto é a leitura bruta, e em janelas maiores, de até dez anos, o gráfico usa médias de 1 min, 10 min, 30 min ou 6 h, com a faixa de mínimo e máximo, para que um pico curto não desapareça. A linha é interrompida onde falta dado. O painel de barra mostra, por intervalo, a média, o mínimo, o máximo, o volume ou o tempo ligado, e o medidor mostra o valor atual dentro de uma faixa.

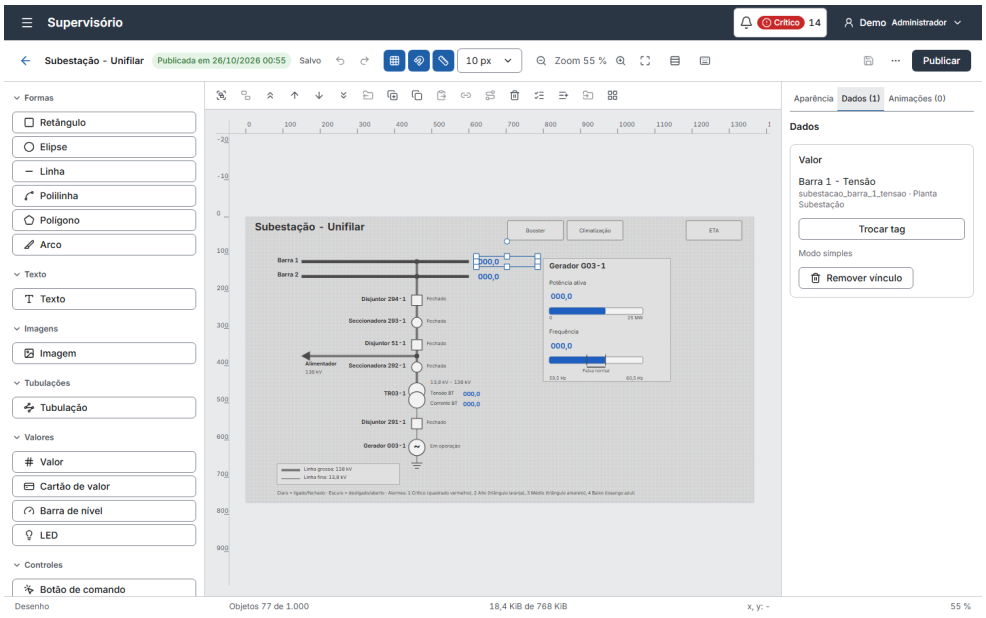
A montagem é feita na tela "Montar dashboard", em que o administrador cria seções e painéis, escolhe o tipo e as tags e ajusta faixas de cor e linhas de meta. Cada tela tem um rascunho e uma versão publicada, e o operador só vê a publicada. Montar telas sem alterar o código cumpre o requisito de incluí-las com o sistema em funcionamento (RF09).

5.4 Sinótico

O Sinótico representa o processo por um desenho vetorial animado pelos dados em tempo real, equivalente moderno do esquema sinótico das salas de controle (RF14). Ele tem uma face de execução, para o operador, e uma de edição, para o administrador.

Na edição, o administrador desenha o processo no navegador, como mostra a Figura 20. À esquerda está a "Paleta", com dezessete tipos de objeto, como tubulações com fluxo animado, cartões de valor, barras de nível, indicadores luminosos e botões. Ao centro fica a área de desenho, e à direita o inspetor do objeto, com as abas "Aparência", "Dados" e "Animações". A biblioteca de imagens fornece figuras de equipamentos.

Figura 20: Editor do sinótico com paleta, área de desenho e inspetor

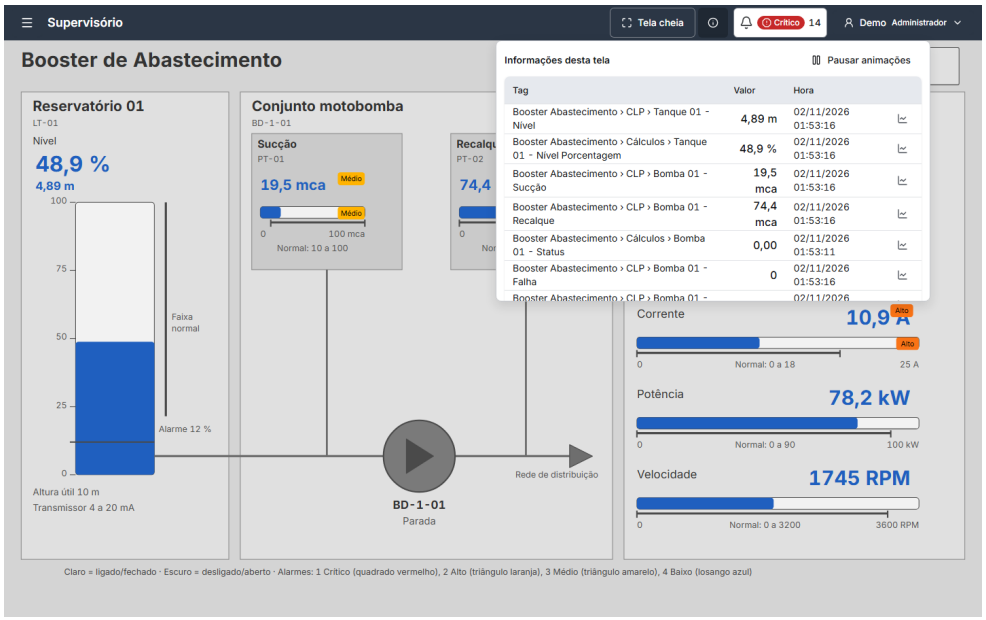


Fonte: Autor.

Na aba “Dados”, uma propriedade do objeto é ligada a uma tag, de forma simples, por tabela (valor ou faixa convertidos em texto ou cor) ou como comando, que abre o diálogo de comando. A aba “Animações” permite cor, visibilidade, preenchimento proporcional, rotação, translação e piscar. Um objeto com defeito aparece como “Objeto inválido”, e o restante da tela continua funcionando.

Na execução, o operador abre o sinótico pelo menu, como mostra a Figura 21. Cada objeto de valor exibe o dado formatado e uma marca de qualidade, e um selo de alarme aparece sobre a tag em alarme. Se não há valor para um objeto em 10 s, ele mostra um traço e “Sem dado”, e o valor desenhado no editor nunca substitui o real. A lista “Informações desta tela” apresenta os mesmos dados em tabela, para quem não usa o desenho. Botões de comando abrem o diálogo de comando, e botões de navegação levam a outras telas.

Figura 21: Sinótico em execução com valores ao vivo, selos de alarme e a lista de valores da tela

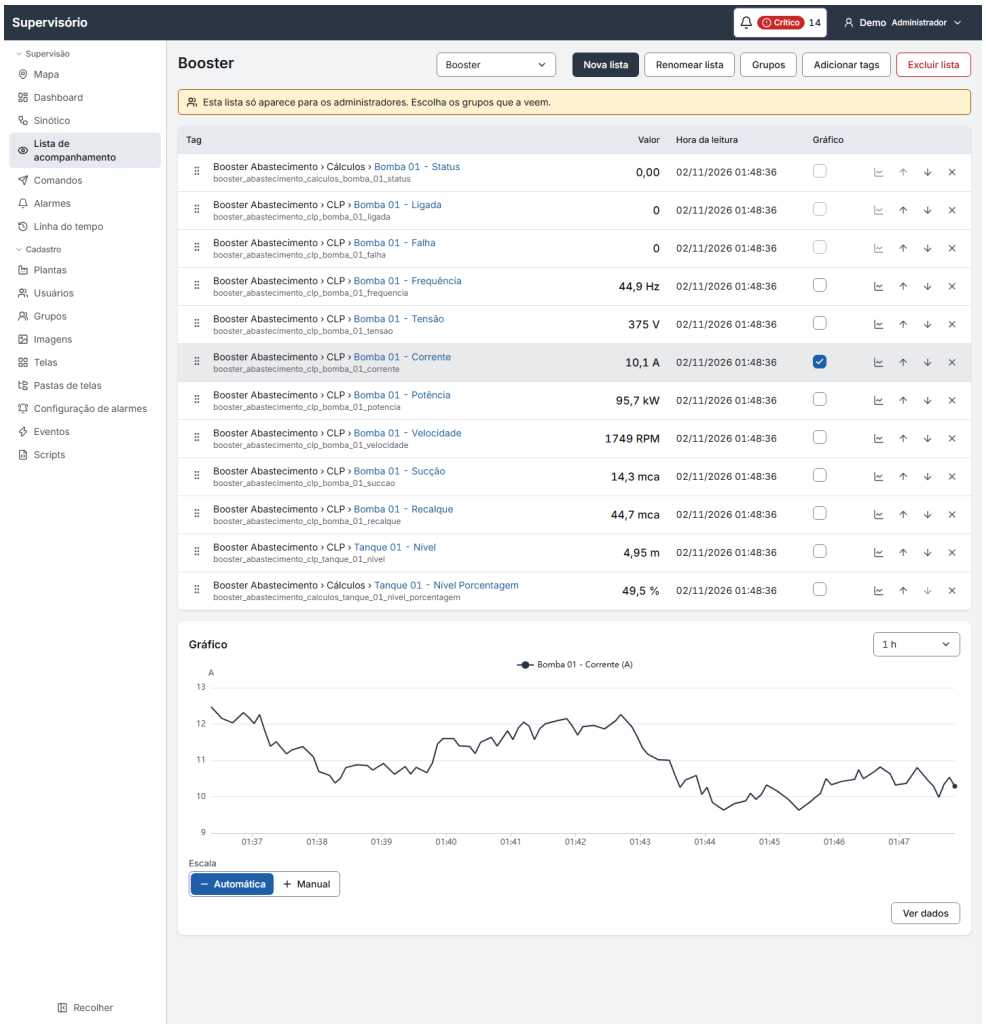


Fonte: Autor.

5.5 Lista de acompanhamento

A Lista de acompanhamento serve para acompanhar poucos pontos de forma contínua, como as listas de observação dos supervisórios clássicos. É um conjunto de até 50 tags, de até 10 plantas, mantido pelo administrador e liberado a grupos, e o operador não pode alterá-lo, como mostra a Figura 22.

Figura 22: Lista de acompanhamento com valores ao vivo e gráfico



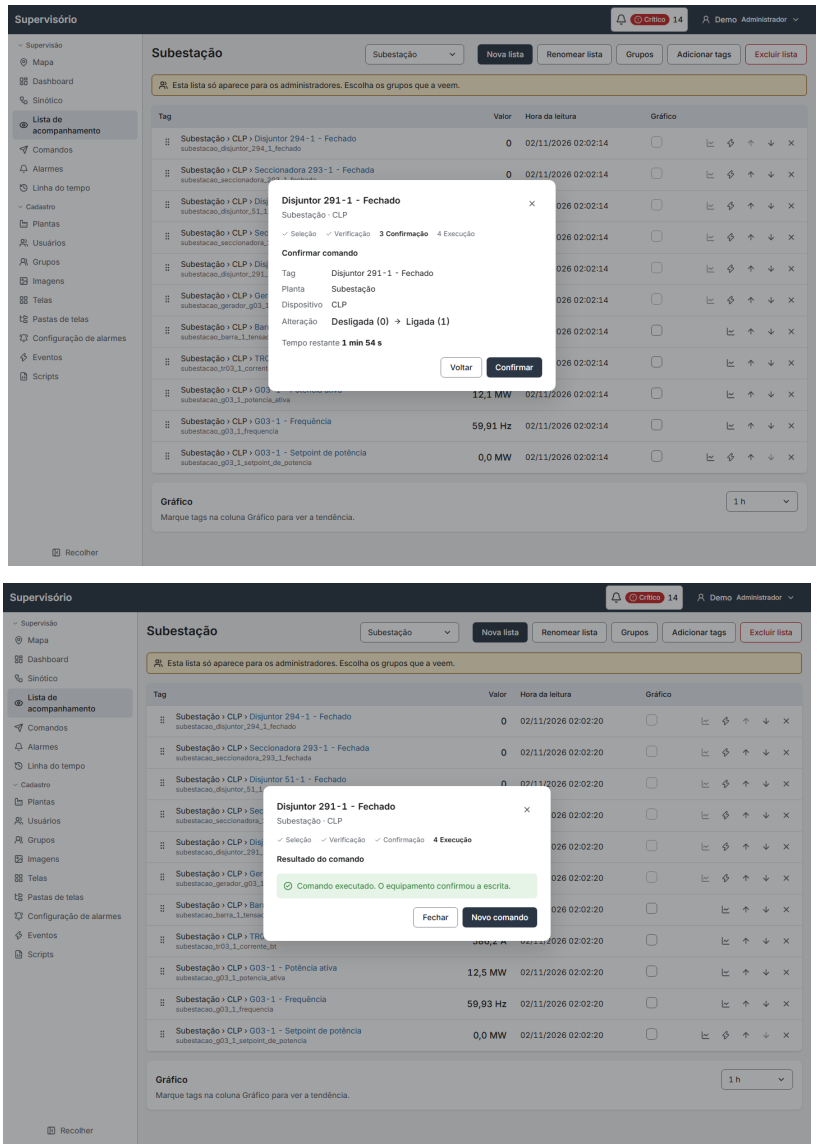
Fonte: Autor.

Cada linha mostra a tag, com a planta e o dispositivo no nome, o valor e a hora da leitura. O valor desatualizado aparece em cinza e itálico, e a falha de comunicação mostra o último valor com um triângulo de alerta. Tags que o usuário pode comandar têm um botão que abre o diálogo de comando, e a coluna “Gráfico” escolhe as tags do painel de tendência abaixo da lista (RF13, RNF01 e RF06).

5.6 Diálogo de comando

O comando é a única ação da interface que escreve no equipamento e passa por um diálogo próprio, usado pela Lista de acompanhamento e pelo Sinótico, como mostra a Figura 23. O operador escolhe “Ligada (1)” ou “Desligada (0)” ou digita o novo valor, revê o resumo e confirma. O comando é enviado ao equipamento e o diálogo mostra o resultado (RF06 e RF08).

Figura 23: Diálogo de comando na confirmação (acima) e no resultado (abaixo)



Fonte: Autor.

O sucesso significa que o equipamento respondeu à escrita com uma resposta que confere com o pedido. Se ele não responde ou responde com erro, o diálogo informa a falha e o motivo. Se o sistema perde o rastro da escrita, a mensagem pede que se confira a lista de comandos antes de repetir, e a tag fica bloqueada por 35 s para evitar uma segunda escrita sobreposta. Esses cuidados correspondem às travas e à auditoria do requisito de segurança (RNF05).

A tela "Comandos" é o histórico dessas ações, com filtros e a opção de cancelar um comando não confirmado. O registro guarda uma cópia dos nomes, de modo que continua legível mesmo que o cadastro mude.

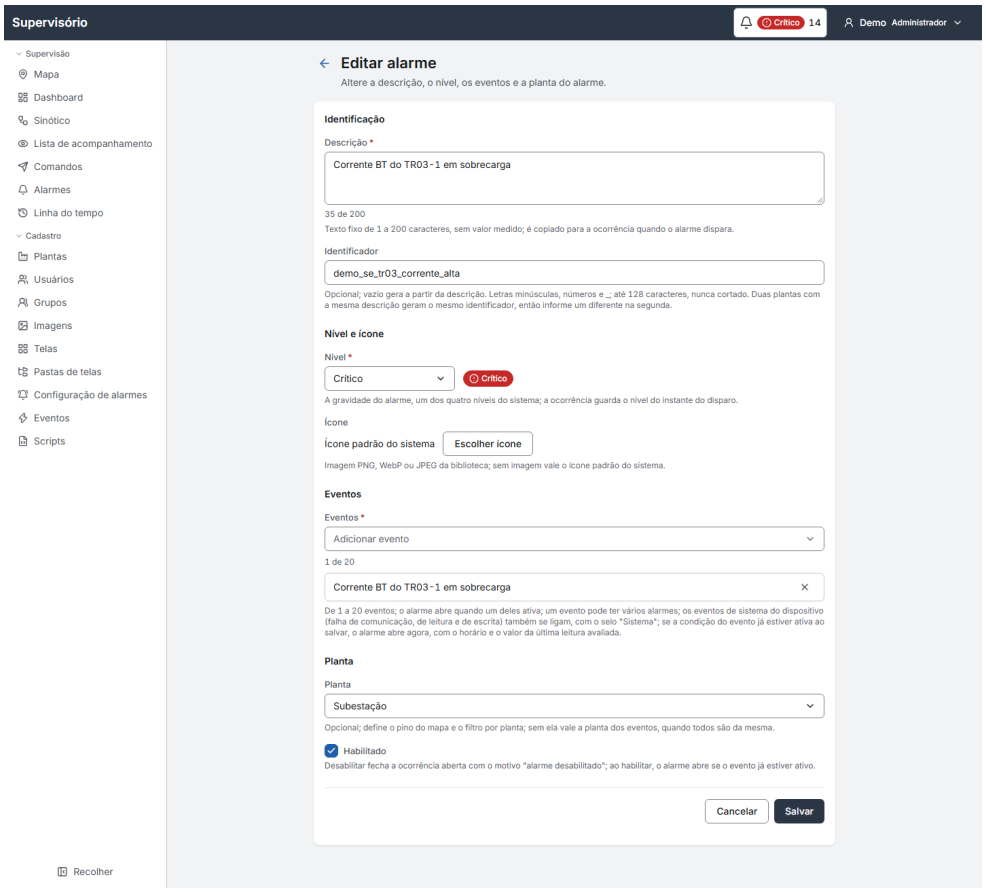
5.7 Alarmes e eventos

O tratamento de alarmes tem telas de configuração, para o administrador, e de operação, para quem acompanha o processo. Os quatro níveis fixos seguem a prática da norma de gestão de alarmes da ISA (International Society of Automation, 2016).

A tela “Eventos” lista as condições, que podem ser de tag, compostas ou agendadas. O evento de tag compara a tag com um valor por um operador, com banda morta e atrasos de ativação e de retorno de 0 a 3600 s. O composto combina eventos com E, OU e NÃO, e o agendado dispara em horários definidos. Cada dispositivo Modbus traz três eventos de sistema, de falha de comunicação, de leitura e de escrita. Um evento, por si só, não gera registro nem aviso.

O alarme é um objeto separado, na tela da Figura 24, com descrição, um dos quatro níveis, ícone, planta e de um a vinte eventos que o abrem. Assim, vários eventos podem acionar o mesmo alarme e um evento pode acionar vários alarmes. O sistema não cria alarmes sozinho: para ver uma falha de comunicação como alarme, liga-se um alarme comum ao evento de sistema (RF16).

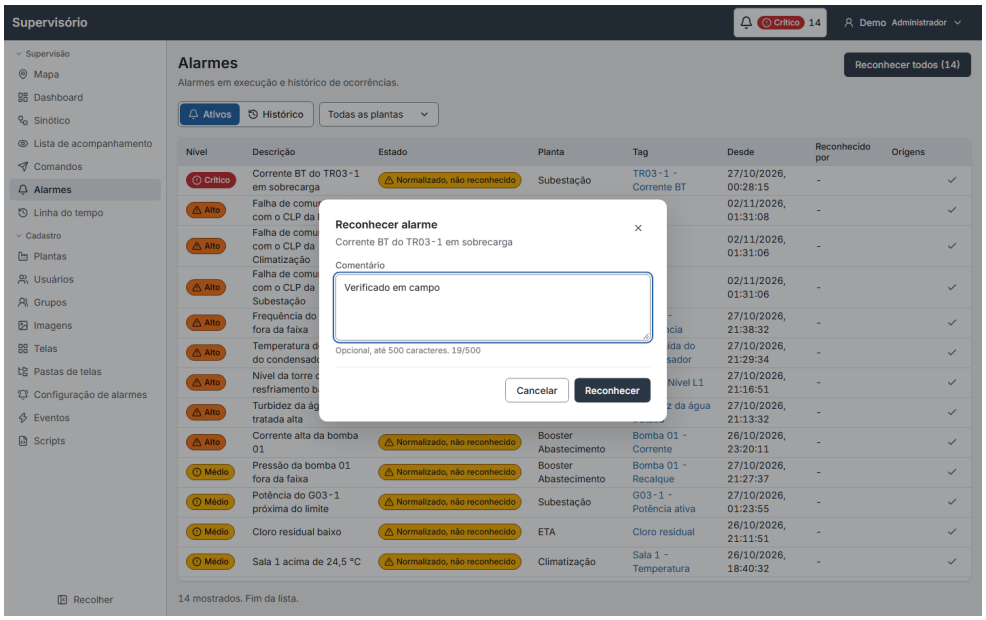
Figura 24: Formulário de configuração de um alarme



Fonte: Autor.

Na operação, a tela “Alarmes”, na Figura 25, tem as abas “Ativos” e “Histórico”. A primeira lista nível, descrição, planta, origens, instante de ativação, estado e quem reconheceu, e a segunda traz as ocorrências fechadas, com duração, motivo e filtros. A ação “Reconhecer” abre um diálogo com comentário opcional, e “Reconhecer todos” pede confirmação. O reconhecimento se refere à ocorrência que a pessoa viu, de modo que um novo disparo não é reconhecido em uma tela velha, e o registro guarda quem reconheceu e quando (RNF05).

Figura 25: Tela de alarmes ativos com reconhecimento

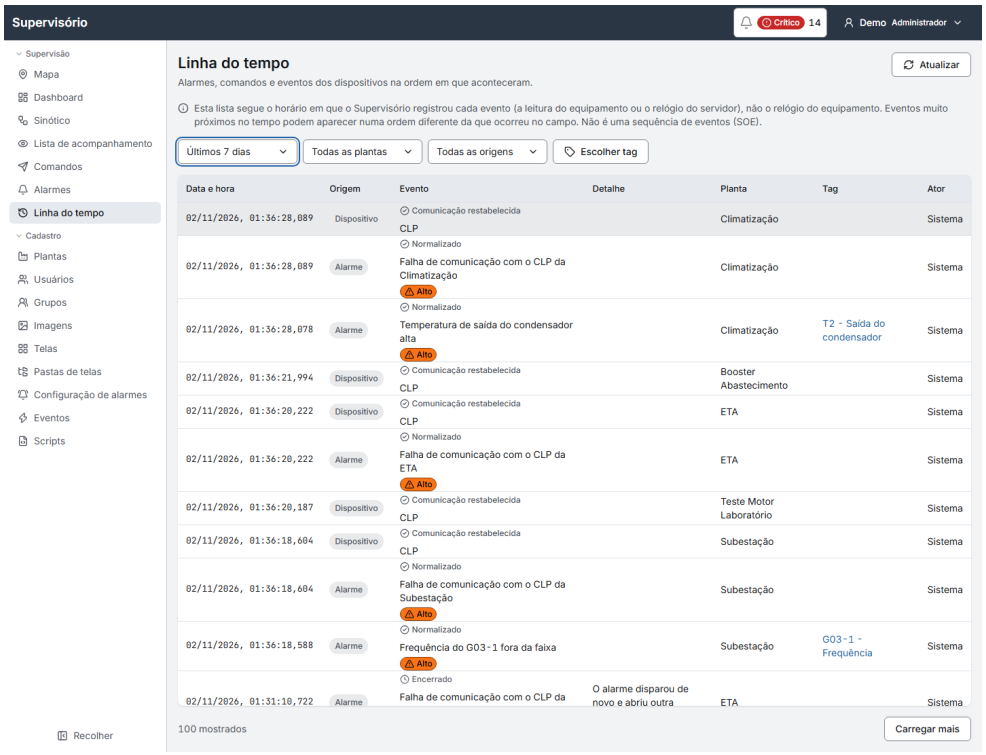


Fonte: Autor.

5.8 Linha do tempo

A Linha do tempo, na Figura 26, é uma lista única, da mais nova para a mais velha, com o que aconteceu com alarmes, comandos e dispositivos. Cada linha traz data e hora, planta, origem, evento e o ator, pessoa ou script. Há filtros por período, de até 31 dias, por planta, por tag e por origem.

Figura 26: Linha do tempo com eventos de alarme e de dispositivo



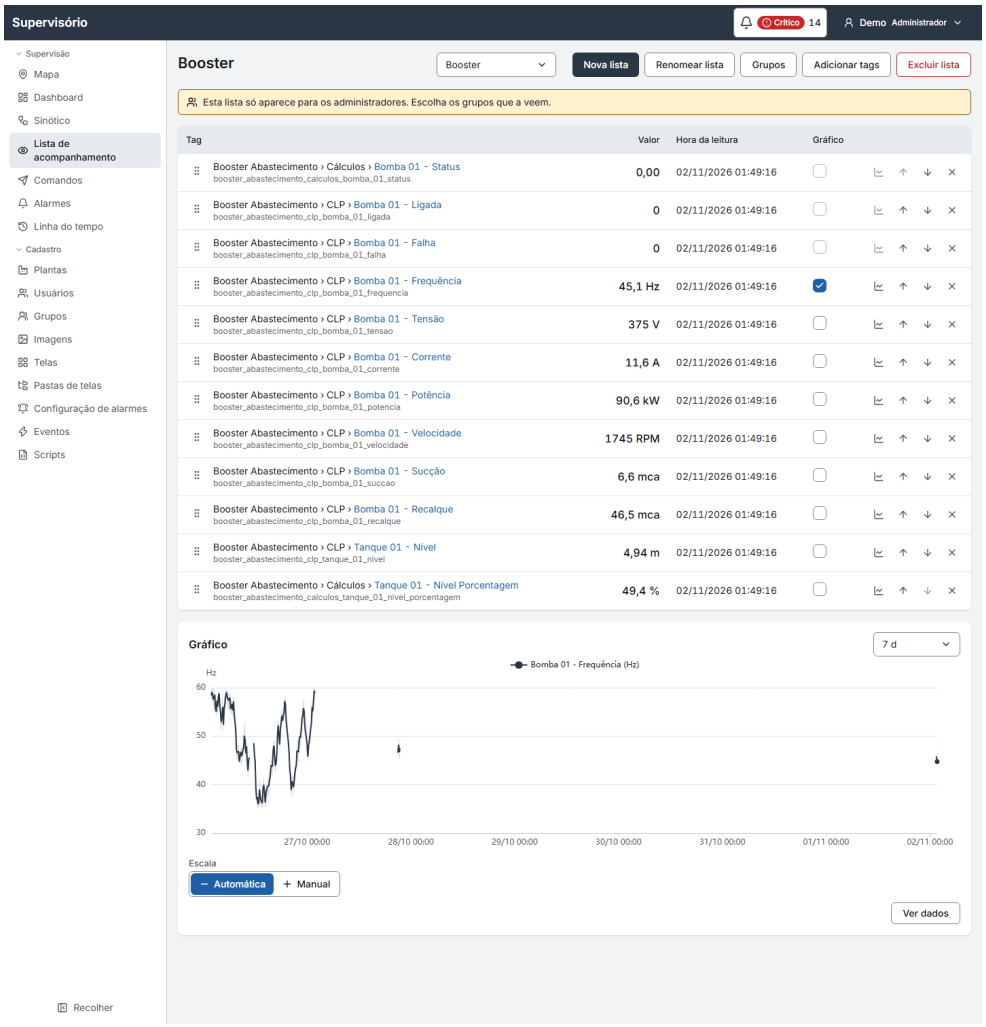
Fonte: Autor.

Cada origem só aparece para quem tem a permissão correspondente. Um aviso informa que a ordem é a do registro do sistema e não a do relógio do equipamento, pois a tela não promete a resolução de uma sequência de eventos com carimbo do equipamento. Esse recurso atende à consulta cronológica (RF17), e a tela de comandos completa a rastreabilidade das ações (RNF05).

5.9 Histórico e tendência

O sistema não tem uma tela isolada de histórico: a consulta está na Lista de acompanhamento e no Dashboard, que usam o mesmo gráfico e a mesma regra de detalhe. A Figura 27 mostra a tendência de uma tag marcada na coluna “Gráfico”.

Figura 27: Tendência de uma tag em janela de sete dias



Fonte: Autor.

O detalhe segue a janela: até uma hora, a leitura gravada, até um dia, a média por minuto, até sete dias, a de dez minutos, até 31 dias, a de trinta minutos, e acima disso a de seis horas, sempre com o mínimo e o máximo do intervalo. Onde falta dado, a linha é interrompida, e a lacuna é a própria evidência de uma falha de leitura. O gráfico reflete o que foi realmente guardado conforme o modo de cada tag (RF02 e RF15).

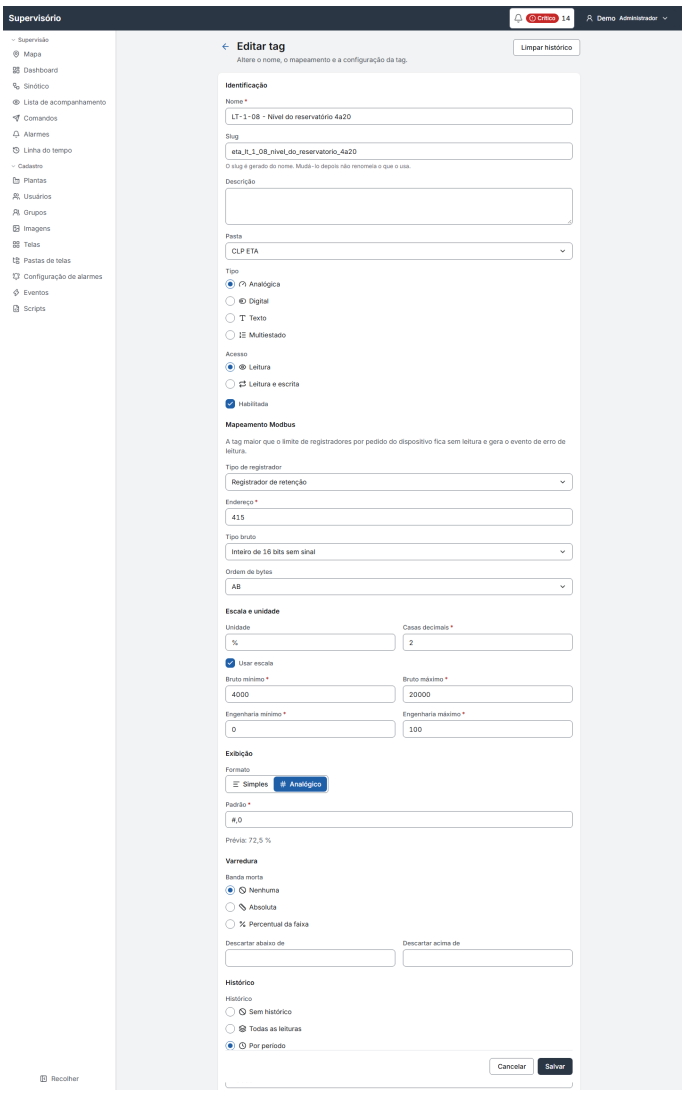
5.10 Cadastro: plantas, dispositivos de E/S, tags e pastas

O cadastro segue a hierarquia planta, dispositivo de E/S e tag, com uma árvore de pastas de tags em cada planta (RF10). Só o administrador da instalação o edita, e toda alteração vale com o sistema em operação (RF09). A tela "Plantas" lista nome, *slug* (identificador textual curto), coordenadas, quantidade de dispositivos e situação, e a exclusão exige digitar o *slug*, já que apaga tudo o que a planta contém.

A lista de "Dispositivos de E/S" mostra nome, protocolo (Modbus ou interno), endereço e situação da comunicação. O formulário do dispositivo Modbus reúne a identificação, a conexão, os tempos e tentativas, os limites do pedido e a escrita (Modbus Organization, 2006a). Como o mapeamento é feito por configuração, equipamentos de fabricantes diferentes entram sem alteração de código (RNF07).

A tela de uma tag, na Figura 28, é dividida em seções. “Identificação” recebe nome, *slug*, pasta, tipo (analógica, digital, texto ou multiestado) e acesso. “Mapeamento Modbus” recebe o tipo de registrador, o endereço, o tipo bruto e a ordem de bytes. “Escala e unidade” define a conversão para a grandeza de engenharia. As demais definem o formato mostrado, a banda morta, o histórico, os eventos de alarme e os limites do comando (RF01 e RF02).

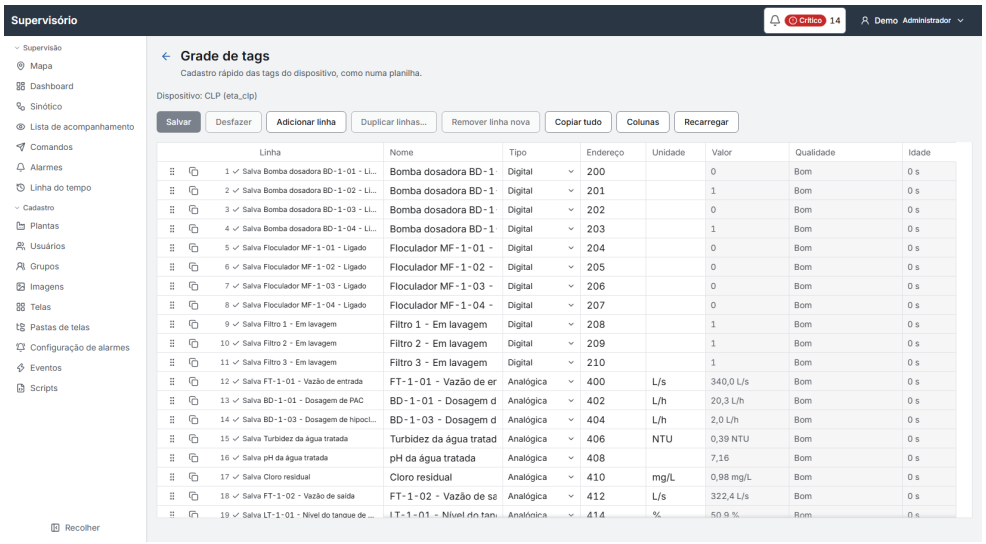
Figura 28: Formulário de uma tag analógica Modbus



Fonte: Autor.

Para cadastrar muitos pontos de uma vez, a ação “Editar em grade”, na Figura 29, apresenta as tags em uma tabela semelhante a uma planilha, que aceita colar blocos do Excel. Células com erro ficam em vermelho e com aviso em amarelo, e o salvamento é tudo ou nada. Três colunas mostram o valor ao vivo, a qualidade e a idade da leitura, o que permite conferir o mapeamento enquanto ele é digitado.

Figura 29: Grade de tags com valor ao vivo



Fonte: Autor.

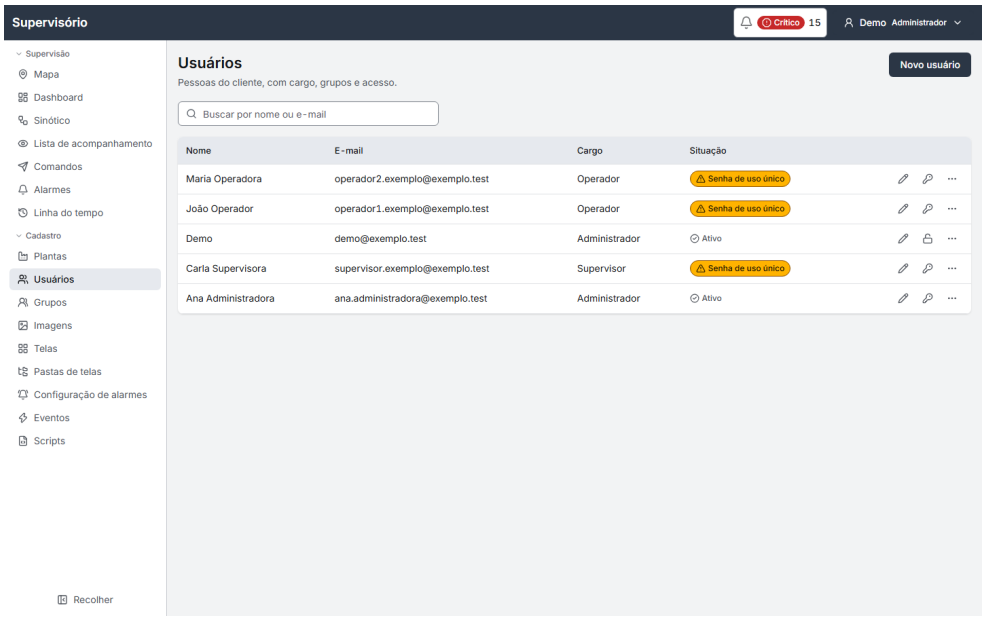
A tela "Pastas" mostra a árvore de pastas da planta. As pastas podem reunir tags de dispositivos diferentes, o que reproduz a organização por equipamento. Elas organizam também a escolha de tags em outras telas.

5.11 Usuários, cargos e grupos

O controle de acesso combina quatro cargos fixos com grupos criados pelo administrador. As permissões vêm de um catálogo de dez itens: ver telas, ver valores das tags, ver histórico, ver alarmes, reconhecer alarme, reconhecer todos os alarmes, ver comandos, comandar estado digital, comandar valor de referência e cancelar comandos (RF05 e RNF04).

A tela "Usuários", na Figura 30, lista nome, e-mail, cargo e situação. O administrador cria usuários de qualquer cargo, e o supervisor gerencia apenas operadores.

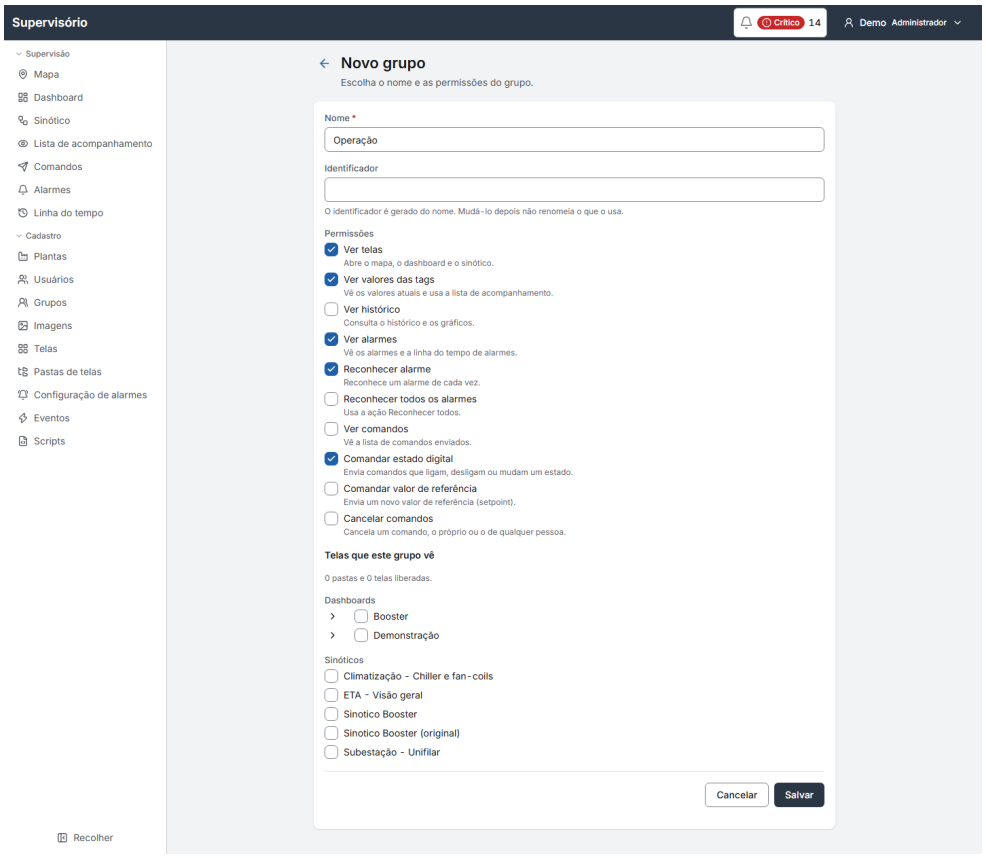
Figura 30: Tela de usuários



Fonte: Autor.

A tela “Grupos”, na Figura 31, lista os grupos, e o formulário apresenta as dez permissões como caixas de seleção, cada uma com uma frase explicando o que libera. A permissão vale para o sistema inteiro, sem filtro por planta, o que simplifica o modelo.

Figura 31: Formulário de grupo com o catálogo de permissões



Fonte: Autor.

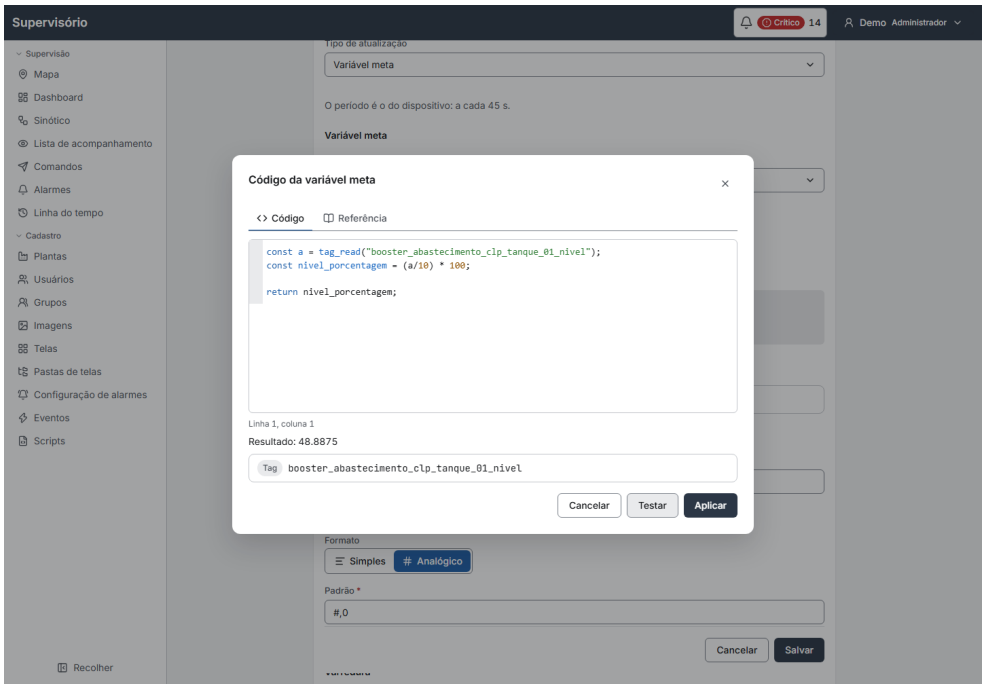
5.12 Variável meta e scripts

O administrador pode escrever pequenos programas em JavaScript que calculam valores e reagem a eventos, executados em um ambiente isolado, sem rede, arquivos ou segredos e com limites de tempo e de memória. O editor é o mesmo nos dois usos, a variável meta e o script.

A variável meta é uma tag de um dispositivo interno calculada a partir de outras tags da mesma planta. A tela da tag mostra o gatilho, por leitura nova ou por tempo, e a ação “Editar código”. O script, configurado na tela “Scripts”, é ligado a eventos, para rodar na ativação, no retorno ou nos dois, e pode ler e escrever em tags e habilitar ou desabilitar tags e dispositivos. A escrita em hora marcada se obtém com um evento agendado, e todo comando de script é registrado com o nome dele.

O editor de código, na Figura 32, tem realce de sintaxe, numeração de linhas e um painel “Referência” com as funções disponíveis. O botão “Testar” confere o código e, na variável meta, mostra o resultado calculado e as tags lidas, e a área “Simular” mostra o que o script faria sem escrever nada. A tela de scripts exibe a “Última execução” e a “Última falha”, mantida mesmo depois de um sucesso, para que um defeito intermitente não passe despercebido.

Figura 32: Editor de código da variável meta com o resultado do botão Testar



Fonte: Autor.

5.13 Síntese: telas, cargos e requisitos

A Tabela 6 reúne as telas, o cargo ou a permissão que dá acesso a cada uma e os requisitos que atendem. As telas de supervisão dependem de permissões concedidas por grupo, e as de configuração dependem do cargo.

Tabela 6: Telas, cargos que acessam e requisitos atendidos.

Tela	**Endereço**	**Quem acessa**	**Requisitos**
Mapa	/mapa	Permissão de ver telas	RF15, RF16
Dashboard (execução)	/dashboard	Permissão de ver telas	RF13, RNF01
Sinótico (execução)	/sinotico, /telas/ver	Permissão de ver telas	RF03, RF06, RF14, RNF01
Montagem de dashboard e do sinótico, telas, pastas de telas e imagens	/telas/..., /configuracao/telas	Administrador da instalação	RF09, RF13, RF14
Lista de acompanhamento	/lista-de-acompanhamento	Permissão de ver valores das tags	RF03, RF06, RF13, RNF01
Comandos (histórico) e diálogo de comando	/comandos	Permissões de ver ou emitir comandos	RF06, RF08, RNF05

Tela	Endereço	Quem acessa	Requisitos
Alarmes (ativos e histórico)	/alarmes	Permissão de ver alarmes (reconhecer exige permissão própria)	RF16, RNF05
Configuração de alarmes e eventos	/configuracao/alarmes, /configuracao/eventos	Administrador da instalação	RF16
Linha do tempo	/linha-do-tempo	Permissão de ver alarmes ou comandos	RF17, RNF05
Plantas, dispositivos, tags e pastas	/cadastro/...	Administrador da instalação	RF01, RF02, RF09, RF10, RNF07
Usuários	/configuracao/usuarios	Administrador da instalação e supervisor (apenas operadores)	RF05
Grupos	/configuracao/grupos	Administrador da instalação	RF05, RNF04
Meu perfil	/meu-perfil	Qualquer usuário autenticado	RF05
Scripts e variável meta	/configuracao/scripts	Administrador da instalação	RF09, RF11, RF12

Fonte: Autor.

O conjunto mostra que a mesma interface atende a quatro perfis de uso. O administrador do sistema e o administrador da instalação configuram o sistema e montam as telas. O supervisor gerencia operadores, e o operador acompanha o processo e emite comandos conforme as permissões do seu grupo. Tudo é acessível por navegador, com atualização em tempo real a partir dos dados do processo.

6 Testes e Resultados

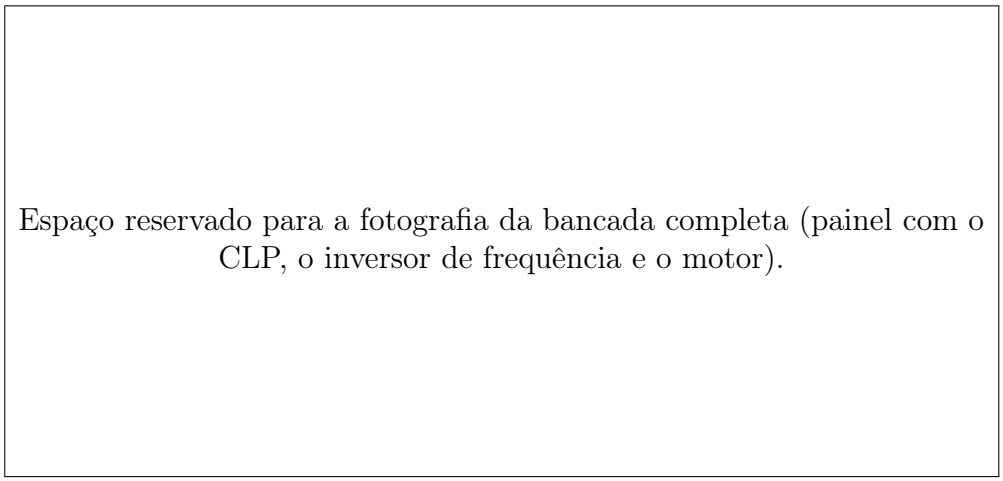
Este capítulo descreve a bancada montada para validar o sistema com equipamentos reais: os equipamentos, a comunicação, a configuração do inversor, o programa do controlador, o cadastro no supervisório, os ensaios e seus resultados.

6.1 Objetivo e Visão Geral

A bancada submete o sistema a um cenário realista, com um controlador lógico programável (CLP), um inversor de frequência e um motor elétrico em funcionamento. O processo é simples: ligar, desligar e mudar a velocidade do motor, que fornece grandezas elétricas reais, como frequência, corrente e tensão.

O supervisório lê essas grandezas e o estado do motor, exibe-os em telas e gráficos, sinaliza alarmes e permite ligar e desligar o motor e alterar a referência de frequência. A Figura 33 reserva o espaço para a foto da bancada.

Figura 33: Bancada de teste montada.



Fonte: Autor.

Inserir a fotografia da bancada montada em Imagens/Desenvolvimento/Cap5/ e trocar o espaço reservado por `\includegraphics`. Informar o local (laboratório do instituto) e a data da montagem.

6.2 Equipamentos Utilizados

A Tabela 7 resume os itens da bancada.

Tabela 7: Equipamentos da bancada de teste.

Equipamento	**Função na bancada**
Servidor	Executa o supervisório
Rede local	Interliga o servidor e o CLP
CLP TM221	Executa a lógica da bancada e responde ao supervisório por Modbus
Inversor ATV320	Aciona o motor

Equipamento	Função na bancada
Motor	Equipamento comandado na bancada

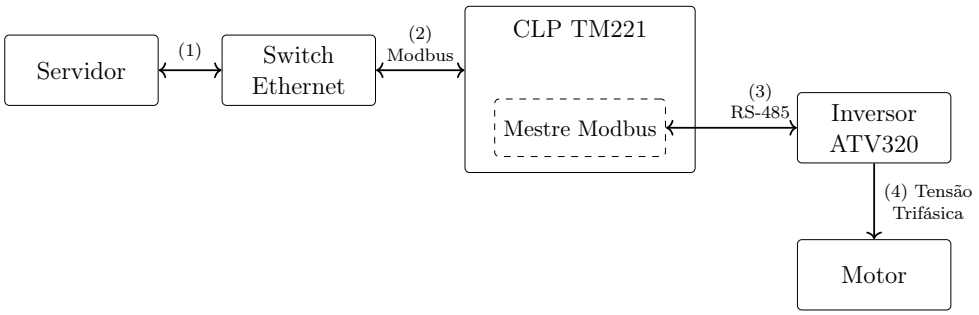
Fonte: Autor.

6.3 Arquitetura de Comunicação

A comunicação tem dois segmentos independentes. No primeiro, a aquisição consulta o CLP por Modbus, pela rede Ethernet (Modbus Organization, 2006a). No segundo, o CLP consulta o inversor por Modbus, em um enlace serial RS-485 (Modbus Organization, 2006b). O CLP é, portanto, servidor para o supervisório e mestre para o inversor, organização encontrada em instalações reais.

Na topologia da Figura 34, o servidor envia requisições Modbus ao CLP pelo switch da rede local (1 e 2), e o CLP responde com os valores em palavras e bits de memória. Em paralelo, o CLP troca dados com o inversor por Modbus em RS-485 e comanda a partida e a parada por uma saída digital (3), e o inversor alimenta o motor (4).

Figura 34: Topologia de comunicação da bancada.



Fonte: Autor.

No segmento Ethernet, as palavras (`%MW`) e os bits de memória (`%M`) são acessíveis pelos mesmos endereços numéricos, contados a partir de zero: `%MW0` é o registrador de retenção de endereço 0 e `%M0` é a bobina de endereço 0 (Schneider Electric, 2026).

No segmento serial, o CLP lê e escreve os registradores do inversor, como mostra a ligação da Figura 35.

Figura 35: Ligação serial RS-485 entre o CLP e o inversor.

Espaço reservado para a fotografia da ligação RS-485 entre o CLP e o inversor.

Fonte: Autor.

6.4 Configuração do Inversor ATV320

O inversor recebe o comando de partida e de parada por uma entrada digital e a referência de frequência pela comunicação Modbus. A Tabela 8 lista os parâmetros.

Tabela 8: Parâmetros do ATV320.

Parâmetro	Valor	Finalidade
Endereço Modbus	2	Identifica o inversor na rede serial
Taxa de transmissão Modbus	19200 bit/s	Velocidade do enlace RS-485
Formato Modbus	8 bits, paridade par, 1 bit de parada (8E1)	Formato padrão do Modbus serial (Modbus Organization, 2006b)
Tempo limite Modbus	10 s	Falha de comunicação se o mestre deixar de consultar
Canal de comando 1	Entrada digital	O comando de marcha e parada vem de uma saída digital do CLP
Canal de referência 1	Modbus	A referência de frequência vem da rede
Rampas de aceleração e de desaceleração	5 s	Suaviza a partida e a parada do motor
Frequência mínima e máxima	0 Hz e 50 Hz	Faixa de operação do motor

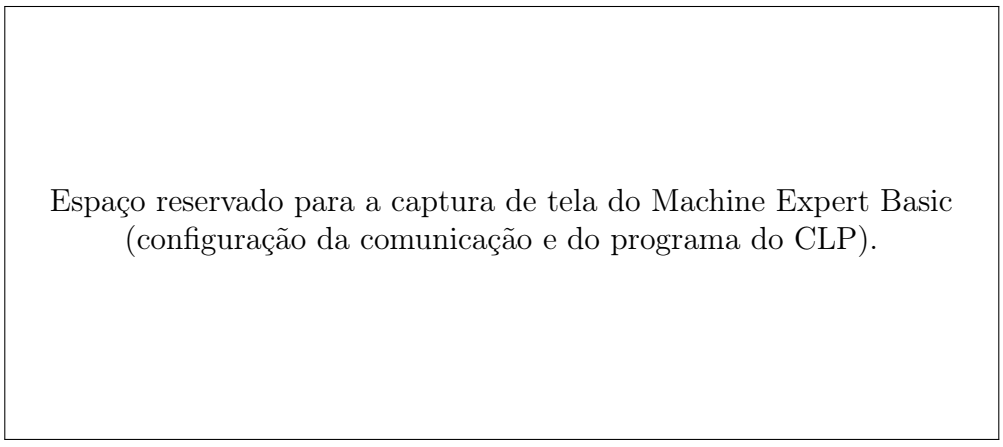
Fonte: Autor.

Conferir os valores da Tabela 8 no inversor.

O CLP lê do inversor a frequência, a corrente, a tensão, a potência e a rotação do motor e escreve a referência de frequência.

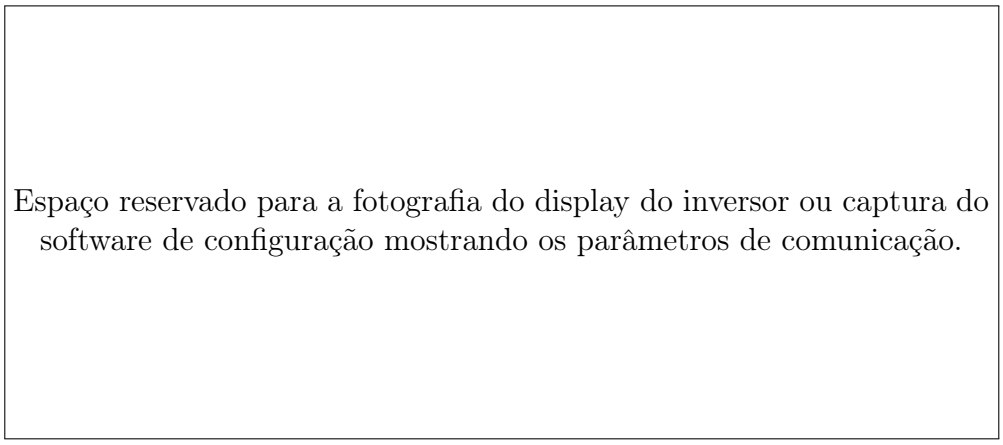
As Figuras 36 e 37 mostram a configuração manual do controlador e do inversor.

Figura 36: Configuração da comunicação serial no EcoStruxure Machine Expert Basic.



Fonte: Autor.

Figura 37: Parametrização do inversor ATV320.

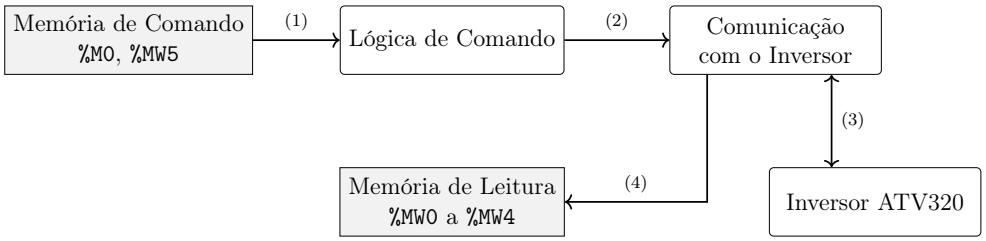


Fonte: Autor.

6.5 Programa do Controlador

O programa do CLP executa, a cada ciclo, a comunicação com o inversor, a lógica de comando e a publicação dos resultados, como mostra a Figura 38.

Figura 38: Blocos funcionais do programa do CLP.



Fonte: Autor.

O supervisório escreve o comando e a referência na memória do controlador (1), e a lógica de comando aciona a saída digital do inversor e envia a referência ao bloco de comunicação (2). O bloco transfere a referência e traz a frequência, a corrente, a tensão, a potência e a rotação (3), publicadas nas posições de leitura (4).

A Tabela 9 reúne o mapa de memória, em que `%M0` é a bobina de endereço 0 e `%MW0` é o registrador de endereço 0. Os valores analógicos são inteiros, e o supervisório reconstrói o valor de engenharia por uma escala linear.

Tabela 9: Mapa de memória do CLP.

Endereço	Tipo	Conteúdo
`%M0`	Bit	Comando liga/desliga escrito pelo supervisório, enviado ao inversor por uma saída digital
`%MW0`	Palavra	Frequência de saída
`%MW1`	Palavra	Corrente do motor
`%MW2`	Palavra	Tensão de saída

Endereço	Tipo	Conteúdo
%MW3	Palavra	Potência
%MW4	Palavra	Rotação (RPM)
%MW5	Palavra	Referência de frequência escrita pelo supervisório

Fonte: Autor.

6.6 Configuração no Supervisório

O cadastro no supervisório é feito pela interface, sem alteração de código.

A planta recebe o nome da bancada e as coordenadas do local. O dispositivo corresponde ao CLP, com protocolo Modbus, endereço IP na rede local, porta 502 e o identificador de unidade do controlador, como resume a Tabela 10.

Tabela 10: Cadastro da bancada no sistema.

Item	Valor
Planta	Bancada do motor
Coordenadas da planta	Latitude −26,8815 e longitude −49,1365 (Instituto Federal Catarinense, Campus Blumenau)
Dispositivo de E/S	CLP TM221, protocolo Modbus
Endereço IP e porta	Endereço do CLP na rede local, porta 502
Identificador de unidade	1
Tempo limite e tentativas	Padrões do sistema (1000 ms e 2 tentativas)

Fonte: Autor.

Cada tag traz o mapeamento Modbus, o tipo bruto e a escala, e o período de varredura é o do dispositivo, de 1 s. Como os valores são inteiros em décimos, a escala linear leva, por exemplo, o intervalo bruto de 0 a 500 ao de 0 a 50 hertz, na frequência. A Tabela 11 apresenta as sete tags.

Tabela 11: Tags cadastradas no sistema para a bancada.

Nome	Endereço Modbus	Tipo bruto	Unidade	Formato
Frequência de saída	Registrador de retenção 0 (%MW0)	Inteiro 16 bits sem sinal	Hz	Analógico
Corrente do motor	Registrador de retenção 1 (%MW1)	Inteiro 16 bits sem sinal	A	Analógico
Tensão de saída	Registrador de retenção 2 (%MW2)	Inteiro 16 bits sem sinal	V	Analógico

Nome	Endereço Modbus	Tipo bruto	Unidade	Formato
Potência	Registrador de retenção 3 (`%MW3`)	Inteiro 16 bits sem sinal	kW	Analógico
Rotação	Registrador de retenção 4 (`%MW4`)	Inteiro 16 bits sem sinal	RPM	Analógico
Comando liga/desliga	Bobina 0 (`%M0`)	Digital, leitura e escrita	n/a	Simples
Referência de frequência	Registrador de retenção 5 (`%MW5`)	Inteiro 16 bits sem sinal, leitura e escrita	Hz	Analógico

Fonte: Autor.

As tags de comando liga/desliga e de referência aceitam escrita, sempre pelo diálogo de comando. Como o comando é bem-sucedido com a resposta do controlador, sem releitura, a confirmação do efeito real vem da leitura posterior da frequência.

Um alarme de corrente alta é configurado sobre a corrente do motor, com limite [A CONFIRMAR] e banda morta, para evitar disparos repetidos, e o evento de falha de comunicação do dispositivo pode ser ligado a um alarme comum.

Confirmar com o autor se o alarme da bancada é o de corrente alta, com o limite, a severidade e a banda morta conforme o motor. Registrar o nome exato dos eventos e dos alarmes criados.

O sinótico representa o motor por um símbolo de onde se abre o diálogo de comando, com a frequência, a corrente, a tensão, a potência e a rotação em indicadores. O *dashboard* reúne um gráfico da frequência e da corrente e um medidor, e a lista de acompanhamento mostra as sete tags. A Figura 39 reserva o espaço para essas telas.

Figura 39: Telas de supervisão da bancada.

Espaço reservado para as capturas do sinótico do motor e do *dashboard* com o gráfico de frequência e corrente.

Fonte: Autor.

Montar o sinótico, o dashboard e a lista de acompanhamento no sistema e inserir as capturas em Imagens/Desenvolvimento/Cap5/. Descrever o desenho do sinótico efetivamente criado (objetos, vínculos de animação e cores).

6.7 Ensaios e Resultados

Seis ensaios exercitam, em sequência, a leitura contínua com o motor desligado, a partida e a parada pelo supervisório, o alarme de corrente alta e seu reconhecimento, a perda e o retorno de comunicação e a troca da referência de frequência. Eles estão na Tabela 12. Duas medidas interessam: o tempo entre a confirmação do comando e a resposta do equipamento, que mede a latência do caminho de escrita, e o atraso do supervisório, entre uma mudança no equipamento e sua exibição na tela, limitado pela soma do período de varredura, do período de atualização da tela e de uma margem de processamento (RNF01).

Tabela 12: Ensaios da bancada.

Id	Ensaio	Requisitos verificados
E1	Leitura contínua com o motor desligado	RF01, RF02, RF03, RF04, RNF01
E2	Partida pelo supervisório	RF06, RF08, RNF01
E3	Parada pelo supervisório	RF06, RF08, RF17, RNF01
E4	Alarme de corrente alta e reconhecimento	RF16, RF17, RNF05
E5	Perda de comunicação e retorno	RF01, RF03, RF16
E6	Troca da referência de frequência	RF06, RF08, RNF05

Fonte: Autor.

Ensaio E1. Com o motor parado, as grandezas não variam, o que isola o desempenho da aquisição. O ensaio mantém as tags em leitura contínua e confere se o histórico não perde amostras, se o carimbo de tempo é monotônico e espaçado pelo período configurado, com variação de poucos milissegundos, e se a lista de acompanhamento atrasa no máximo o período de varredura mais o de atualização da tela. Espera-se uma taxa de entrega próxima de 100 % e nenhum alarme.

Ensaio E2. O operador define a referência de 40 hertz e envia o comando de partida. O sistema escreve a ordem em uma bobina do CLP, que aciona o inversor por uma saída digital. O ensaio mede o tempo entre a confirmação e a resposta do CLP, em que o comando é dado como concluído, e o tempo até a frequência começar a subir na tela. Espera-se a resposta em dezenas de milissegundos, bem abaixo do limite de 30 s do comando, que a trava por tag impeça um segundo comando simultâneo e que o comando fique registrado com o operador, a tag, o valor e o instante.

Ensaio E3. A ordem de parada segue o mesmo caminho, e a frequência desce pela rampa de desaceleração. Espera-se o mesmo tempo de comando do E2, que a tela acompanhe a frequência até zero e que a leitura continue sem perda, sem alarme de comunicação, o que separa o efeito do processo do efeito do sistema.

Ensaio E4. Com o motor ligado, a corrente cruza o limite configurado [A CONFIRMAR] do alarme de corrente alta, e o detector cria uma ocorrência. O ensaio mede o tempo entre o cruzamento e a exibição no indicador de alarmes, que deve ser da ordem do atraso do supervisório. O operador reconhece o alarme, e confere-se que o registro guarda o nome e o instante.

Parado o motor, a corrente cai abaixo do limite, descontada a banda morta, e o alarme retorna ao normal. Espera-se um alarme único por cruzamento e a sequência correta na linha do tempo.

Ensaio E5. A comunicação é interrompida desconectando o cabo Ethernet entre o comutador e o CLP, e depois restabelecida. Durante a falha, o CLP continua executando o programa. O sistema marca o valor como desatualizado depois de três períodos de varredura, mantendo o último valor e o carimbo, e o dispositivo passa a degradado depois de três ciclos falhos. Com o cabo de volta, o primeiro ciclo bem-sucedido fecha o estado, sem rajada de leituras acumuladas. O ensaio mede o tempo até a marcação de desatualizado e o tempo até a volta à qualidade boa e confere que não há lacuna preenchida no histórico, pois a lacuna é a evidência da falha.

Ensaio E6. Com o motor a 40 hertz, o operador altera a referência para 30 hertz pelo mesmo comando. O ensaio mede o tempo até a resposta do CLP e até a frequência atingir a referência, e confere que um segundo comando imediato é recusado pela trava por tag e que o reenvio do mesmo pedido não cria uma segunda escrita.

A Tabela 13 reúne as medidas previstas.

Tabela 13: Medidas previstas nos ensaios da bancada.

Ensaio	**Medida**	**Esperado**	**Medido**
E1	Taxa de leituras gravadas em relação às esperadas	acima de 99 %	a medir
E1	Variação do intervalo entre carimbos consecutivos	poucos ms	a medir
E1	Atraso do supervisório (equipamento até a tela)	varredura mais atualização da tela	a medir
E2	Confirmação do operador até a resposta do CLP à escrita	dezenas de ms	a medir
E2	Resposta do CLP até a primeira mudança de frequência na tela	varredura mais atualização da tela	a medir
E2	Duração da rampa de aceleração observada	valor configurado no inversor	a medir
E3	Confirmação do operador até a resposta do CLP à escrita	dezenas de ms	a medir
E3	Resposta do CLP até a frequência zero na tela	rampa de desaceleração mais atraso do supervisório	a medir
E3	Leituras perdidas durante a transição	nenhuma	a medir
E4	Cruzamento do limite até o alarme na tela	varredura mais atualização da tela	a medir
E4	Ocorrências geradas por cruzamento	uma	a medir

Ensaio	Medida	Esperado	Medido
E4	Reconhecimento até o registro com nome e instante	imediato (próxima atualização da tela)	a medir
E5	Interrupção até a marcação de desatualizado	três períodos de varredura	a medir
E5	Interrupção até o estado de comunicação degradado	três ciclos falhos	a medir
E5	Retorno do cabo até a qualidade boa na tela	um ciclo mais atualização da tela	a medir
E5	Linhas gravadas no histórico durante a falha	nenhuma	a medir
E6	Confirmação do operador até a resposta do CLP à escrita	dezenas de ms	a medir
E6	Resposta do CLP até a frequência medida igual à referência	rampa do inversor mais atraso do supervisório	a medir
E6	Segundo comando imediato na mesma tag	recusado pela trava	a medir
E6	Reenvio do mesmo pedido	uma única escrita	a medir

Fonte: Autor.

Conferir os parâmetros citados (referência de 40 e 30 Hz, rampas do inversor, limite de corrente) e medir e inserir, para cada ensaio, o gráfico da tela e os valores da Tabela 13. Informar as rampas do inversor, o registrador da referência, o período de varredura, a corrente em cada regime e a linha do registro de comandos. Confirmar a referência de 40 e 30 Hz e se o alarme de falha de comunicação foi ligado.

7 Conclusão

Os supervisórios convencionais exigem licenças proprietárias, equipamentos dedicados e instalação especializada, o que os afasta de pequenas e médias empresas (Zanghi, 2019; Phuyal *et al.*, 2020). A resposta proposta foi um supervisório com arquitetura orientada a serviços, em contêineres, acessado pelo navegador. O objetivo geral foi atendido na medida descrita a seguir: o sistema foi construído por inteiro, executa em um único servidor e foi exercitado por testes automatizados, e os ensaios com equipamentos reais estão descritos no Capítulo 6.

7.1 Retomada dos objetivos

O primeiro objetivo específico era revisar os conceitos da solução, reunidos no Capítulo 2: supervisórios, Indústria 4.0, nuvem, contêineres, bancos de dados, filas, o protocolo Modbus e a execução isolada (Bangemann *et al.*, 2014; Lasi *et al.*, 2014).

O segundo era especificar os requisitos e projetar a arquitetura. O Capítulo 3 fixou dezessete requisitos funcionais e dez não funcionais e mostrou como cada módulo de um supervisório é atendido pelo *front-end*, pelo *back-end*, pelos *workers*, pela aquisição, pelo executor de scripts, pelo PostgreSQL e pelo Redis.

O terceiro era implementar a aquisição via Modbus, com o registro do valor atual e do histórico. O Capítulo 4 descreveu o caminho do dado, e o ensaio E1 do Capítulo 6 confere a leitura contínua e o histórico.

O quarto era a interface de supervisão, apresentada no Capítulo 5, com um conjunto único de componentes.

O quinto era o comando de equipamentos, implementado com trava por tag, idempotência e auditoria, com o sucesso definido pela resposta do controlador. Os testes automatizados exercitam a cadeia, mas a validação com o equipamento real depende dos ensaios de bancada, e por isso o requisito está parcialmente atendido.

O sexto era o controle de acesso, com quatro cargos fixos e grupos de permissões, coberto por testes e por um teste de segurança de caixa preta.

O sétimo era validar o sistema em uma bancada com um CLP e um inversor de frequência. O Capítulo 6 descreveu a bancada, com um CLP TM221 e um inversor ATV320, e os seis ensaios. Os ensaios de bancada ainda dependem da execução pelo autor.

7.2 Contribuições

A primeira contribuição é um supervisório completo, de código próprio, executável em um computador comum. A segunda é o comando de equipamentos com garantias verificáveis por teste.

A terceira é a execução isolada do código do usuário, em um interpretador sem rede, arquivos ou segredos (Haas *et al.*, 2017).

7.3 Limitações

O efeito de réplicas não foi medido, e a redundância entre servidores não foi implantada.

Seis requisitos estão parcialmente atendidos: RF06, RF08, RF16 e RNF01 dependem dos ensaios com a bancada, e RNF02 e RNF03 dependem de medidas com mais de uma réplica e de ensaio de queda de réplica.

Fechar esta seção depois dos ensaios E1 a E6 da bancada: informar quais requisitos passam de parcial para atendido, os tempos de resposta do comando e o atraso do supervisório medidos, e ajustar a frase sobre o objetivo de validação em bancada.

7.4 Trabalhos futuros

O principal caminho é levar o sistema a um provedor de nuvem e a mais de um servidor, sem alterar o código, o que permitiria medir o efeito das réplicas e tolerar a queda do servidor que hoje guarda o banco e o Redis, avaliada com injeção de falhas.

A arquitetura da aquisição também abre espaço para outros protocolos industriais. Ficam ainda como evolução relatórios, notificações por canais externos, restrição de acesso por planta, um aplicativo móvel e os ensaios de bancada com cargas mais variadas e redes de campo.

O trabalho entrega uma base funcional e verificável para um supervisório orientado a serviços acessível a empresas que não podem arcar com a solução tradicional, e a arquitetura em contêineres permite evoluí-la sem reescrever o que já foi construído.

Referências

ABBAS, Hosny Ahmed. **Efficient Web-Based SCADA System**. 2011. Dissertação (Mestrado em Engenharia Elétrica) – Faculty of Engineering, South Valley University, Aswan.

BANGEMANN, Thomas *et al.* State of the Art in Industrial Automation. *In:* COLOMBO, Armando W. *et al.* (ed.). **Industrial Cloud-Based Cyber-Physical Systems**: The IMC-AESOP Approach. Cham: Springer, 2014. p. 23–47. ISBN 978-3-319-05623-4. DOI: 10.1007/978-3-319-05624-1_2.

BIGHETI, Jeferson André. **Arquitetura de Automação e Controle Orientada a Microserviços para a Indústria 4.0**. 2020. Tese (Doutorado em Engenharia Elétrica) – Faculdade de Engenharia, Universidade Estadual Paulista "Júlio de Mesquita Filho", Bauru.

DOCKER, INC. **Docker Documentation**. Docker, Inc. Disponível em: https://docs.docker.com/. Acesso em: 16 out. 2026.

ELIPSE SOFTWARE. **Elipse E3 completa 20 anos**. Elipse Software. 9 jun. 2021. Disponível em: https://www.elipse.com.br/produtos/elipse-e3-completa-20-anos/. Acesso em: 2 nov. 2026.

HAAS, Andreas *et al.* Bringing the Web up to Speed with WebAssembly. *In:* PROCEEDINGS of the 38th ACM SIGPLAN Conference on Programming Language Design and Implementation. [*S. l.: s. n.*], 2017. p. 185–200. DOI: 10.1145/3062341.3062363.

HENRIQUES DA SILVA, Fabrício Rodrigues. **Um estudo sobre os benefícios e os riscos de segurança na utilização de Cloud Computing**. 2010. Trabalho de Conclusão de Curso (Graduação em Ciência da Computação) – Centro Universitário Augusto Motta, Rio de Janeiro.

INTERNATIONAL SOCIETY OF AUTOMATION. **Enterprise-Control System Integration. Part 1: Models and Terminology**. Research Triangle Park, NC, 2000. ISBN 1-55617-727-5.

INTERNATIONAL SOCIETY OF AUTOMATION. **Management of Alarm Systems for the Process Industries**. Research Triangle Park, NC, 2016.

LASI, Heiner *et al.* Industry 4.0. **Business & Information Systems Engineering**, v. 6, n. 4, p. 239–242, 2014. DOI: 10.1007/s12599-014-0334-4.

META PLATFORMS, INC. **React Documentation**. Meta Platforms, Inc. Disponível em: https://react.dev/. Acesso em: 16 out. 2026.

MICROSOFT. **TypeScript Documentation**. Microsoft. Disponível em: https://www.typescriptlang.org/docs/. Acesso em: 16 out. 2026.

MODBUS ORGANIZATION. **Modbus Messaging on TCP/IP Implementation Guide**. Versão 1.0b. Hopkinton, MA, 24 out. 2006. Disponível em:

https://www.modbus.org/file/secure/messagingimplementationguide.pdf. Acesso em: 2 nov. 2026.

MODBUS ORGANIZATION. **Modbus over Serial Line Specification and Implementation Guide**. Versão 1.02. Hopkinton, MA, 20 dez. 2006. Disponível em: https://www.modbus.org/file/secure/modbusoverserial.pdf. Acesso em: 2 nov. 2026.

MOHAMED, Mohamed A.; ALTRAFI, Obay G.; ISMAIL, Mohammed O. Relational vs. NoSQL Databases: A Survey. **International Journal of Computer and Information Technology**, v. 3, n. 3, p. 598–601, 2014.

MOLECULERJS. **Moleculer Documentation**. MoleculerJS. Disponível em: https://moleculer.services/docs/0.14/. Acesso em: 14 jun. 2026.

MONOSTORI, L. *et al.* Cyber-physical systems in manufacturing. **CIRP Annals**, v. 65, n. 2, p. 621–641, 2016. DOI: 10.1016/j.cirp.2016.06.005.

NETTO, Hylson V. *et al.* State machine replication in containers managed by Kubernetes. **Journal of Systems Architecture**, 2016. DOI: 10.1016/j.sysarc.2016.12.007.

NETTO, Hylson Vescovi *et al.* Coordenação de Containers no Kubernetes: Uma Abordagem Baseada em Serviço. *In:* SIMPÓSIO Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC). [*S. l.*]: Sociedade Brasileira de Computação, 2017.

OPENJS FOUNDATION. **Fastify Documentation**. Fastify. Disponível em: https://fastify.dev/docs/latest/. Acesso em: 16 out. 2026.

OPENJS FOUNDATION. **Node.js Documentation**. Node.js. Disponível em: https://nodejs.org/docs/latest/api/. Acesso em: 16 out. 2026.

PEDROSA, Paulo Henrique C.; NOGUEIRA, Tiago. Computação em Nuvem. Trabalho acadêmico (disciplina G04). [*S. l.*], 2011.

PHUYAL, Sudip *et al.* Design and Implementation of Cost Efficient SCADA System for Industrial Automation. **International Journal of Engineering and Manufacturing**, v. 10, n. 2, p. 15–28, 2020. DOI: 10.5815/ijem.2020.02.02.

PISCHING, Marcos André. **Arquitetura para descoberta de equipamentos em processos de manufatura com foco na indústria 4.0**. 2018. Tese (Doutorado em Engenharia de Controle e Automação Mecânica) – Escola Politécnica, Universidade de São Paulo, São Paulo.

POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. PostgreSQL Global Development Group. Disponível em: https://www.postgresql.org/docs/current/. Acesso em: 14 jun. 2026.

REDIS LTD. **Redis Documentation**. Redis Ltd. Disponível em: https://redis.io/docs/latest/. Acesso em: 14 jun. 2026.

SCHNEIDER ELECTRIC. **EcoStruxure Machine Expert - Basic: Modicon M221 Programming Guide**. [*S. l.*]. Disponível em: https://www.se.com/br/pt/. Acesso em: 16 out. 2026.

STANKOVIC, John A. Research Directions for the Internet of Things. **IEEE Internet of Things Journal**, v. 1, n. 1, p. 3–9, 2014. DOI: 10.1109/JIOT.2014.2312291.

STOUFFER, Keith *et al.* **Guide to Industrial Control Systems (ICS) Security**. Gaithersburg, MD, maio 2015. Retirada em 28 set. 2023 e substituída pela Revisão 3. DOI: 10.6028/NIST.SP.800-82r2. Disponível em: https://nvlpubs.nist.gov/nistpubs/specialpublications/nist.sp.800-82r2.pdf. Acesso em: 13 jun. 2026.

TASKFORCE.SH INC. **BullMQ Documentation**. Taskforce.sh Inc. Disponível em: https://docs.bullmq.io/. Acesso em: 16 out. 2026.

VERCEL, INC. **Next.js Documentation**. Vercel, Inc. Disponível em: https://nextjs.org/docs. Acesso em: 14 jun. 2026.

YADAV, Geeta; PAUL, Kolin. **Architecture and Security of SCADA Systems: A Review**. arXiv:2001.02925 [cs.CR]. arXiv. 9 jan. 2020. Disponível em: https://arxiv.org/abs/2001.02925. Acesso em: 13 jun. 2026.

ZANGHI, Eric. **Sistemas SCADA: Conceitos**. Porto, Portugal, 2019. Série Proteção e Comunicação de Sistemas Elétricos de Potência (PROTCOM).